



Zachodniopomorski
Uniwersytet Technologiczny
w Szczecinie



Wydział
Informatyki

inż. Mikołaj Sitek

nr albumu: 49395

kierunek studiów: informatyka

specjalność: systemy komputerowe zorientowane na człowieka

forma studiów: studia stacjonarne II stopnia

**System automatycznego wspomagania triażu w urazach i
problemach zdrowotnych w obrębie klatki piersiowej z
wykorzystaniem algorytmów sztucznej inteligencji**

**A system for automatic triage supporting in thoracic injuries
and health problems using artificial intelligence algorithms**

praca dyplomowa magisterska

napisana pod kierunkiem:

dr hab. inż. Anny Lewandowskiej, prof. ZUT

Katedra Systemów Multimedialnych

Szczecin, 2025

System automatycznego wspomagania triażu w urazach i problemach zdrowotnych w obrębie klatki piersiowej z wykorzystaniem algorytmów sztucznej inteligencji

inż. Mikołaj Sitek

Promotor: dr hab. inż. Anna Lewandowska, prof. ZUT

Streszczenie

Niniejsza praca dyplomowa przedstawia projekt i implementację systemu informatycznego wspomagającego triaż medyczny oraz wstępną diagnostykę chorób klatki piersiowej. Celem było stworzenie narzędzia wspierającego personel medyczny podczas kwalifikacji pacjentów w warunkach przedszpitalnych i ambulatoryjnych oraz jako platformy edukacyjnej.

System został zbudowany w architekturze MEAN (MongoDB, Express.js, Angular, Node.js) i działa całkowicie lokalnie w przeglądarce, bez potrzeby połączenia z Internetem. Gwarantuje to wysoki poziom prywatności i możliwość pracy w środowiskach o ograniczonym dostępie do sieci. Interfejs opiera się na dynamicznym formularzu objawowym, w którym użytkownik wprowadza symptomy i podstawowe parametry życiowe.

Algorytm regułowy klasyfikuje poziom pilności (triazu) zgodnie z pięciostopniową skalą ESI. Dodatkowy moduł dopasowania objawów umożliwia identyfikację prawdopodobnych diagnoz ICD-10 na podstawie indeksu Jaccarda, porównując zestawy objawów pacjenta z danymi referencyjnymi.

Walidację systemu przeprowadzono na 698 formularzach treningowych i 200 testowych. Trafność klasyfikacji triażu osiągnęła 90 % zgodności z ocenami lekarzy, a skuteczność predykcji chorób wyniosła 40 % (top-1) i 60 % (top-3). Badanie użyteczności z udziałem personelu medycznego ($n=7$) dało wynik $SUS = 82/100$, co potwierdziło intuicyjność systemu.

Wskazano także możliwe kierunki rozwoju, w tym integrację z algorytmami uczenia maszynowego, rozszerzenie bazy wiedzy, obsługę standardów HL7/FHIR oraz zastosowanie w rozwiązaniach telemedycznych. System ten stanowi podstawę dla dalszego rozwoju narzędzi wspierających decyzje kliniczne.

Słowa kluczowe: triage, SOR, klatka piersiowa, system wspomagania decyzji medycznych, predykcja choroby, aplikacja webowa, MEAN stack, uczenie maszynowe, ICD-10, algorytmy regułowe

A system for automatic triage supporting in thoracic injuries and health problems using artificial intelligence algorithms

inż. Mikołaj Sitek

Supervisor: dr hab. inż. Anna Lewandowska, prof. ZUT

Abstract

This engineering thesis presents the design and implementation of an IT system supporting medical triage and preliminary diagnosis of chest diseases. The aim was to create a tool that assists healthcare staff in patient prioritization in pre-hospital and outpatient settings, and also serves as an educational platform.

The system is built using the MEAN stack (MongoDB, Express.js, Angular, Node.js) and runs entirely offline in the user's browser, ensuring high data privacy and usability in limited-network environments. The interface is based on a dynamic symptom form that allows the user to enter key symptoms and vital signs.

A rule-based algorithm determines the triage level according to the five-level Emergency Severity Index (ESI). Additionally, an optional symptom-matching module estimates the most probable ICD-10 diagnoses using the Jaccard index to compare patient input with reference datasets.

Validation was conducted on 698 training and 200 test forms, with 90 % triage accuracy compared to physicians' assessments. Disease prediction reached 40 % accuracy for top-1 results and 60 % for top-3. Usability testing with medical personnel (n=7) yielded a SUS score of 82/100, confirming the system's intuitiveness.

The thesis also outlines development directions, including integration with machine learning, expansion of the symptom-diagnosis database, HL7/FHIR support, and telemedicine applications. The system lays the groundwork for modern clinical decision-support tools in resource-constrained settings.

Keywords: triage, emergency department, chest pain, medical decision support system, disease prediction, web application, MEAN stack, machine learning, ICD-10, rule-based algorithms

Spis treści

Spis rysunków	5
Spis tabel	6
Spis listingów	7
Wprowadzenie	8
1 Przegląd istniejących rozwiązań w dziedzinie związanej z tematem pracy	9
1.1 Kontekst kliniczny i skala problemu	9
1.2 Istota i znaczenie triage'u w medycynie	10
1.3 Przykładowe metody i skale triage'u	10
1.4 Patofizjologia klatki piersiowej i najczęstsze stany nagłe	12
1.5 Procedura ABCDE — krok po kroku	12
1.6 Porównanie skal pięciostopniowych	13
1.7 Regulacje prawne i standardy bezpieczeństwa	13
1.8 Systemy wspomagania decyzji w medycynie	14
1.9 Wnioski	14
2 Opis metody rozwiązania problemu prezentowanego w pracy	16
2.1 Ogólny opis metody	16
2.1.1 Przykładowy przebieg metody krok po kroku	16
2.1.2 Zbiory danych	18
2.1.3 Porównanie z istniejącymi rozwiązaniami	20
2.1.4 Charakterystyka interfejsów	20
2.1.5 Analiza wymagań i założeń projektowych	21
2.1.6 Projektowanie systemu	21
2.1.6.1 Ograniczenia i założenia	21
2.1.6.2 Bezpieczeństwo i prywatność	22
2.2 Technologie i narzędzia	22
2.2.1 Architektura MEAN	22
2.2.2 Baza danych MongoDB	22
2.2.3 Serwer Node.js/Express i REST API	23
2.2.4 Angular – moduły i biblioteki	23
2.3 Podsumowanie	24

3	Opis implementacji metody	25
3.1	Opis środowiska	25
3.2	Format danych wejściowych i wyjściowych	25
3.3	Implementacja systemu	27
3.3.1	Makieta w Figmie – szybka walidacja układu	27
3.3.2	Struktura projektu i kluczowe widoki interfejsu	28
3.3.2.1	Kluczowe fragmenty front-endu	36
3.3.2.2	Wycinek szablonu Angular (HTML)	36
3.3.2.3	Fragment stylów (CSS)	37
3.3.2.4	Przykład logiki (TypeScript)	37
3.3.2.5	Uruchomienie obliczeń i obsługa loadera	38
3.3.3	Warstwa serwera i punkty końcowe API	39
3.3.4	Mechanizm szybkiego triage'u	40
3.3.5	Moduł lokalnej predykcji choroby	41
3.4	Podsumowanie	42
4	Rezultaty i wnioski	43
4.1	Opis procedury testowej	43
4.1.1	Cel i zakres badań	43
4.1.2	Charakterystyka danych	43
4.1.3	Metryki oceny	44
4.1.4	Procedura badawcza	44
4.2	Prezentacja rezultatów	44
4.2.1	Rozkład zgodności objawów	44
4.2.2	Trafność priorytetu triage	45
4.2.3	Analiza źródeł błędu	46
4.2.4	Rozkład błędów w priorytecie triage	46
4.2.5	Analiza czasowo-wydajnościowa	47
4.2.6	Ocena użyteczności interfejsu	47
4.3	Podsumowanie	48
5	Możliwości rozwoju i rozszerzenia systemu	49
5.1	Integracja z zewnętrznymi bazami wiedzy medycznej (API, standardy ICD)	49
5.2	Rozbudowa algorytmów sztucznej inteligencji (np. uczenie maszynowe)	49
5.3	Dodatkowe moduły (konsultacje online, telemedycyna)	50
5.4	Skalowanie systemu w środowiskach chmurowych	50
5.5	Podsumowanie	51
	Podsumowanie	52

Spis rysunków

1.1	Wzrost liczby wizyt w SOR w UE i Polsce (2012–2022).	10
1.2	Przykładowa krzywa ROC i wartość $AUC = 0,90$ dla modelu przewidującego potrzebę przyjęcia na SOR.	11
2.1	Trzy równorzędne zbiory danych wykorzystywane w pracy.	19
3.1	Tablica robocza w Figmie – wszystkie komponenty i ekrany prototypu.	27
3.2	Widok kroku 2: formularz zgłaszania objawów (makieta Figmy).	27
3.3	Widok kroku 3: historia medyczna i parametry życiowe (makieta Figmy).	28
3.4	Lokalizacja dolegliwości – wybór obszaru bólu.	28
3.5	Walidacja obszaru – potwierdzenie lokalizacji bólu.	29
3.6	Objawy główne – opis bólu, duszności i kaszlu.	29
3.7	Objawy towarzyszące – objawy towarzyszące i czas nawrotu kapilarnego.	30
3.8	Wywiad medyczny – historia medyczna i parametry życiowe.	31
3.9	Ekran oczekiwania – trwa lokalna analiza danych.	32
3.10	Wynik niebieski – brak potrzeby interwencji medycznej.	33
3.11	Wynik zielony – niskie ryzyko, zalecana konsultacja POZ.	33
3.12	Wynik żółty – umiarkowane ryzyko, pilna konsultacja.	34
3.13	Wynik pomarańczowy – wysokie ryzyko i pytanie o możliwość dojazdu.	34
3.14	Mapa najbliższych oddziałów ratunkowych po wyborze opcji „Tak”.	35
3.15	Wynik czerwony – stan krytyczny, natychmiastowe wezwanie 112.	36
4.1	Rozkład współczynnika Jaccarda w zbiorze testowym.	45
4.2	Macierz pomyłek klasyfikacji priorytetu triage.	45
4.3	Histogram rozkładu różnic poziomów w klasyfikacji priorytetu triage (absolutna różnica pomiędzy etykietą systemu i lekarza).	46
4.4	Mediana czasu odpowiedzi $\pm$ IQR (interquartile range).	47
4.5	Średnie oceny 10 pytań SUS.	48

Spis tabel

1.1	Porównanie kluczowych cech i skuteczności skal triage	13
2.1	Dane wejściowe pacjenta #A-057	17
2.2	Priorytet i rozpoznanie dla pacjenta #A-057 – porównanie lekarz vs system	18
3.1	Pięć rang priorytetu i odpowiadające im zalecenia.	32
3.2	Przykładowe przedziały punktowe dla poszczególnych kategorii triage .	41
4.1	System Usability Scale – treść 10 pytań (skala 1–5)	47

Spis listingów

2.1	Fragment żądania JSON	17
2.2	Fragment odpowiedzi JSON	18
3.1	Przykładowe żądanie JSON do endpointu <code>/api/triage</code>	25
3.2	Przykładowa odpowiedź serwera	26
3.3	Fragment HTML – wybór obszaru ciała	36
3.4	Fragment CSS – interakcje obszaru klikalnego	37
3.5	Fragment TS – Metoda <code>onSelect()</code> i getter <code>selectedAreaName</code>	37
3.6	Asynchroniczne obliczenia i loader w kroku 5	38
3.7	Inicjalizacja Express i definicja API w <code>server.js</code>	39

Wprowadzenie

Tematem pracy dyplomowej magisterskiej jest stworzenie systemu informatycznego umożliwiającego szybką ocenę stanu pacjenta oraz predykcję choroby na podstawie analizy danych wejściowych. Praca osadzona jest w obszarze medycyny ratunkowej i systemów wspomagania decyzji, wykorzystujących nowoczesne technologie informatyczne.

Kluczowym pojęciem wykorzystywanym w pracy jest *triage* – proces szybkiej oceny stanu zdrowia pacjenta, który umożliwia ustalenie priorytetu interwencji medycznych, co jest szczególnie istotne w sytuacjach kryzysowych. Równie ważnym zagadnieniem jest *predykcja choroby*, czyli oszacowanie ryzyka wystąpienia określonego stanu patologicznego na podstawie analizowanych danych medycznych. System informatyczny definiowany jest jako zintegrowany zbiór narzędzi i aplikacji służących automatycznemu przetwarzaniu i analizie danych.

Poniższa praca powstała z potrzeby znalezienia odpowiedzi na problem przeciążonych SOR-ów, który wynika z nieodpowiedniej klasyfikacji pacjentów i ich dolegliwości. Często osoby nieznające charakteru swoich objawów trafiają na SOR, nawet gdy ich stan nie wymaga pilnej interwencji. Taka sytuacja prowadzi do blokowania miejsc przeznaczonych dla pacjentów z poważnymi urazami lub stanami zagrażającymi życiu, co w konsekwencji opóźnia udzielanie pomocy tym, którzy jej najbardziej potrzebują. Zaproponowany system jest rozwiązaniem, które umożliwia precyzyjną ocenę stanu pacjenta oraz właściwe skierowanie go do odpowiedniej jednostki opieki – poradni, SOR-u lub, w przypadku braku konieczności interwencji, do samodzielnego monitorowania stanu zdrowia.

Celem pracy jest opracowanie oraz implementacja systemu informatycznego, który dzięki wykorzystaniu technologii takich jak platforma MEAN (MongoDB, Express, Angular, Node.js) oraz dedykowanych algorytmów predykcyjnych umożliwia szybką ocenę stanu zdrowia pacjenta. W ramach realizacji tego celu stworzono interaktywną aplikację, w której pacjent wprowadza dane dotyczące swoich objawów, a system na ich podstawie generuje rekomendacje dotyczące dalszego postępowania medycznego.

Teza pracy: Jeżeli system wykorzysta nowoczesne technologie informatyczne oraz odpowiednio skonstruowane algorytmy predykcyjne, możliwe będzie znaczące usprawnienie procesu triage'u, co przełoży się na odciążenie SOR-ów i poprawę jakości opieki medycznej, umożliwiając szybsze udzielanie pomocy pacjentom wymagającym pilnej interwencji.

W dalszej części pracy przedstawiono kolejno: przegląd literatury wraz z kontekstem medycznym, analizę wymagań i założeń projektowych, opis zastosowanych technologii i narzędzi, szczegółowy projekt systemu wraz z implementacją oraz podsumowanie przeprowadzonych testów wraz z wnioskami końcowymi.

Rozdział 1

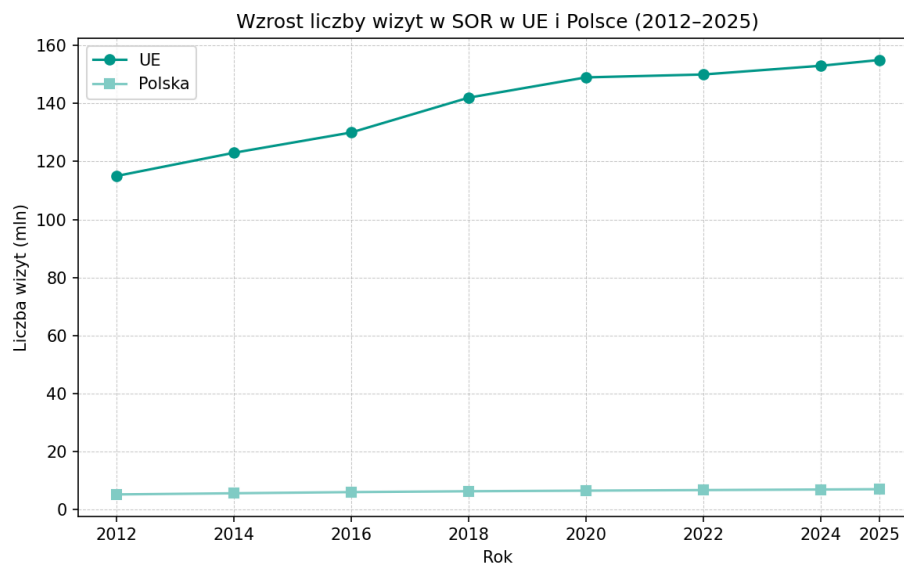
Przegląd istniejących rozwiązań w dziedzinie związanej z tematem pracy

1.1 Kontekst kliniczny i skala problemu

W ostatniej dekadzie oddziały ratunkowe w Europie i Ameryce Północnej odnotowały znaczący wzrost liczby pacjentów zgłaszających się z nagłymi przypadkami. Według danych European Society for Emergency Medicine (EUSEM) liczba wizyt w szpitalnych oddziałach ratunkowych (SOR) w Unii Europejskiej rosła w latach 2012–2022 o średnio 3–5 % rocznie, osiągając blisko 150 mln konsultacji rocznie [SHB19]. Równocześnie w Polsce w 2023 r. przyjęto ponad 6,7 mln chorych, z czego ok. 30–35 % mogło zostać obsłużonych w podstawowej opiece zdrowotnej lub nocnej i świątecznej pomocy lekarskiej. Dane te ukazują powszechne występowanie crowding’u czyli przeciążenia oddziałów ratunkowych, który prowadzi do:

- wydłużenia średniego czasu oczekiwania powyżej 160 minut,
- wzrostu odsetka zdarzeń niepożądanych o 14–20 %,
- zwiększenia śmiertelności wewnątrzszpitalnej o 1,5 p.p. w środowiskach o skrajnym przepełnieniu.

Dlatego odciążenie SOR [LTH18] oraz optymalizacja procesu triage’u, zarówno pod kątem szybkości, jak i dokładności oceny, wskazują ogromny potencjał, który stał się jednym z priorytetów polityk zdrowotnych w UE i USA.



Rysunek 1.1: Wzrost liczby wizyt w SOR w UE i Polsce (2012–2022).

1.2 Istota i znaczenie triage’u w medycynie

Triage (z francuskiego *trier* – sortować) to procedura wstępnej oceny medycznej, której zadaniem jest ustalenie priorytetów interwencji dla zgłaszających się pacjentów. Jego rozwój sięga konfliktów zbrojnych XIX w., jednak do szerszego zastosowania w codziennej praktyce klinicznej trafił wraz z rozwojem standardów SOR w latach 70. i 80. XX w. Najważniejsze funkcje triage’u to:

- identyfikacja pacjentów w stanie bezpośredniego zagrożenia życia,
- efektywna alokacja ograniczonych zasobów (personelu, sal, sprzętu),
- zapewnienie równowagi pomiędzy pilnymi i mniej pilnymi przypadkami.

Badania wielośrodkowe potwierdzają, że dobrze przeprowadzony triage skraca czas do pierwszej konsultacji medycznej nawet o 30–40 % w porównaniu do systemów nieuporządkowanych [FJS10]. Jednak błędy undertriage (zbyt niski priorytet) i overtriage (zbyt wysoki priorytet) nadal stanowią istotne wyzwanie organizacyjne i kliniczne [MJMD97].

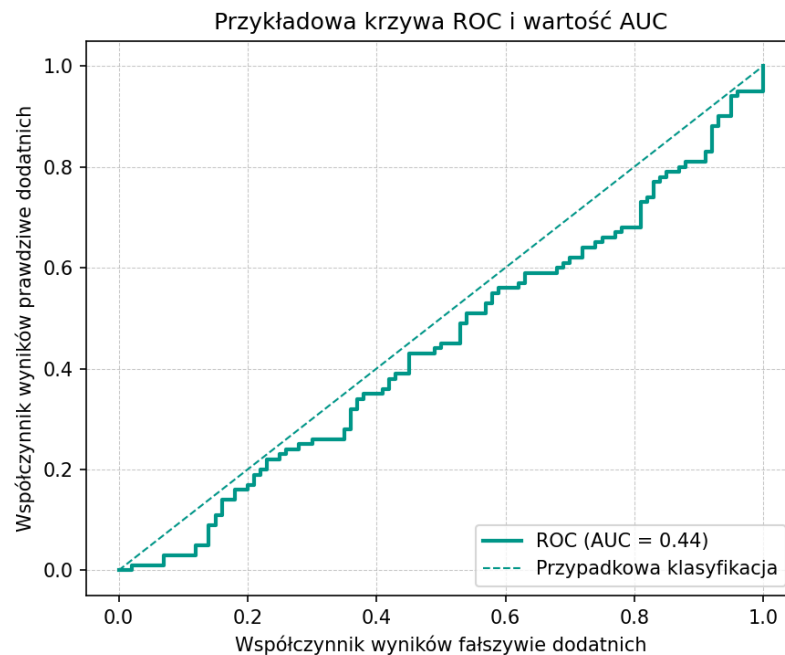
1.3 Przykładowe metody i skale triage’u

Współczesne systemy triage’u opierają się niemal wyłącznie na pięciostopniowych skalach, które – mimo wspólnej struktury – różnią się szczegółowymi kryteriami. W Europie dominującym rozwiązaniem jest Manchester Triage System (MTS), w którym czerwony kolor oznacza stan absolutnego zagrożenia życia, a na każdym poziomie definiuje się maksymalny czas do udzielenia pomocy oraz zestaw objawów alarmowych. Dla personelu pielęgniarskiego MTS jest czytelny i intuicyjny, co przekłada się na skrócenie czasu oceny oraz zmniejszenie liczby błędów undertriage [MJMD97].

W Stanach Zjednoczonych powszechnie stosuje się Emergency Severity Index (ESI), który poza oceną pilności bierze pod uwagę także liczbę zasobów diagnostyczno-terapeutycznych potrzebnych do opieki nad pacjentem. Dzięki temu ESI dobrze prognozuje konieczność

przyjęcia na SOR – w badaniach modele oparte na ESI osiągały wartość AUC rzędu 0,85–0,90, przewyższając wiele klasycznych skal [GTR12].

AUC (Area Under the Curve) to miara pola pod krzywą ROC (Receiver Operating Characteristic), obrazująca zdolność modelu do rozróżniania między dwoma klasami (tu: potrzeba przyjęcia na SOR vs. brak takiej potrzeby). Wartość 1,0 oznacza perfekcyjną klasyfikację, 0,5 – losowy wybór, a każda wyższa wartość AUC świadczy o lepszej ogólnej skuteczności modelu.



Rysunek 1.2: Przykładowa krzywa ROC i wartość $AUC = 0,90$ dla modelu przewidującego potrzebę przyjęcia na SOR.

W Kanadzie obowiązuje Canadian Triage and Acuity Scale (CTAS), w której każde z pięciu poziomów ma ściśle określony maksymalny czas oczekiwania (od natychmiastowego miejsca w kolejce do 120 minut). CTAS kładzie silny nacisk na wieloaspektową ocenę ryzyka klinicznego, co pozwala na precyzyjne dostosowanie priorytetów do lokalnej organizacji pracy szpitala [BUSG14].

Z kolei w Australii i Nowej Zelandii używa się Australian Triage Scale (ATS), która łączy detekcję objawów krytycznych z elastyczną oceną personelu medycznego. ATS definiuje kryteria tak, by uwzględnić zarówno subiektywną ocenę lekarza, jak i obiektywne miary stanu pacjenta, co zapewnia wysoki poziom akceptacji wśród użytkowników oraz dobrą trafność klasyfikacji [CR19].

Porównując te cztery skale, można stwierdzić, że wszystkie osiągnęły zbliżone wartości czułości i swoistości. ESI wyróżnia się jednak w kontekście decyzji o przyjęciu na SOR, MTS natomiast zdobywa przewagę prostotą i przejrzystością kryteriów, a CTAS zapewnia doskonałe dopasowanie czasowe do zasobów szpitalnych. ATS z kolei stanowi kompromis między automatyzacją a doświadczeniem personelu, co czyni go uniwersalnym rozwiązaniem w wielu różnych warunkach klinicznych.

1.4 Patofizjologia klatki piersiowej i najczęstsze stany nagłe

Klatka piersiowa chroni narządy o kluczowym znaczeniu życiowym – płuca, serce, duże naczynia i przeponę. W przebiegu ostrych chorób lub stanów pourazowych mogą jednak wystąpić mechanizmy bezpośredniego zagrożenia życia. Najczęściej obserwowane na SOR (szpitalnym oddziale ratunkowym, ang. emergency department – ED) stany nagłe obejmują:

- **Tamponadę osierdzia** – krwawienie do worka osierdziowego ograniczające wypełnianie serca; śmiertelność 50–60 % w ciągu 1–2 h,
- **Odę prężną** – uwięzione powietrze w jamie opłucnej przesuwające śródpiersie; 30–40 % zgonów w kilka minut,
- **Masywny hemothorax** – >1500 ml krwi w opłucnej i wstrząs hipowolemiczny; 40–60 % w ciągu 2 h,
- **Flail chest** – wieloodłamowe złamania żeber z paradoksalnym ruchem segmentu i hipowentylacją; 15–20 % w pierwszych dobach,
- **Ostry STEMI/NSTEMI** – zakrzep tętnicy wieńcowej wywołujący zawał; 7–15 % do wypisu,
- **Masywną zatorowość płucną** – skrzeplinę w pniu płucnym prowadzącą do ostrej niewydolności prawej komory; 25–30 % w pierwszej godzinie.

Ich wczesne rozpoznanie opiera się na szybkim badaniu fizykalnym (triada Becka, osłuchiwanie, ocena perfuzji) oraz badaniach obrazowych (USG FAST, RTG) [Szc24].

Zagrożenia te są identyfikowane w praktyce przy pomocy uniwersalnego algorytmu **ABCDE**, który przedstawiony został w dalszej części pracy.

1.5 Procedura ABCDE — krok po kroku

Procedura **ABCDE** (ang. *Airway, Breathing, Circulation, Disability, Exposure*) to standardowy algorytm wstępnej oceny pacjenta w stanie krytycznym [Szc24], którego poszczególne litery przypisane są do konkretnych objawów. Poniżej przedstawiono jego pięć kroków; dla każdej litery wyszczególniono kluczowe objawy i progi, które natychmiast podnoszą priorytet triage'u:

- A – Airway** sprawdzenie drożności dróg oddechowych; wystąpienie stridoru, chrapania lub całkowitej niedrożności wymaga natychmiastowego udrożnienia (manewr Esmarcha, przyrządy nadgłośniowe) bądź intubacji.
- B – Breathing** ocena wentylacji i utlenowania; saturacja < 90 % lub częstość oddechów > 30/min są uznawane za *czerwone flagi* i nadają najwyższy priorytet.
- C – Circulation** ocena perfuzji; tętno > 120/min, ciśnienie < 90/60 mmHg albo czas powrotu kapilarnego > 2 s sugerują wstrząs hipowolemiczny i wymagają natychmiastowej płynoterapii.
- D – Disability** szybka ocena neurologiczna (AVPU/GCS); GCS < 13 pkt lub objawy ogniskowe oznaczają zagrożenie OUN i wskazują na pilne badanie obrazowe.

E – Exposure całkowite odsłonięcie pacjenta, zabezpieczenie przed hipotermią i identyfikacja urazów ukrytych (otarcia, krwiaki, ciała obce).

Po zabezpieczeniu ABCDE pacjent jest formalnie klasyfikowany w jednej z pięciu kategorii pilności stosowanych w międzynarodowych skalach triage'u.

1.6 Porównanie skal pięciostopniowych

Pięciostopniowe skale triage'u stanowią obecnie międzynarodowy standard w oddziałach ratunkowych, ponieważ łączą szybkie podejmowanie decyzji z wystarczającą rozdzielczością kliniczną, aby wyraźnie odróżnić stany natychmiastowego zagrożenia życia od przypadków stabilnych. Każdy system wykorzystuje pięć poziomów pilności, lecz różni się algorytmem przypisywania priorytetu — od czysto objawowych reguł (MTS) po modele, które dodatkowo szacują zużycie zasobów diagnostyczno-terapeutycznych (ESI) lub precyzyjnie definiują limity czasowe i kryteria ryzyka (CTAS). W krajach regionu Azji i Pacyfiku popularność zyskał natomiast ATS, który integruje parametry życiowe z oceną kliniczną personelu, zapewniając dużą elastyczność w różnych warunkach organizacyjnych. Poniżej zestawiono najważniejsze cechy każdego z tych czterech systemów:

- **MTS** – algorytm objawowy, czasy maksymalne (0, 10, 60, 120, 240 min),
- **ESI** – oprócz pilności ocenia liczbę potrzebnych zasobów (badania, RTG, TK),
- **CTAS** – precyzyjnie określone limity czasowe i szerokie kryteria ryzyka,
- **ATS** – łączy parametry życiowe z oceną personelu, zapewniając większą elastyczność.

Skala	Czas i poziom	Zasoby?	AUC przyjęcia	Undertriage
MTS	< 2 min	nie	0,82	7–9 %
ESI	< 2 min	tak	0,85–0,90	5–7 %
CTAS	< 1 min	częściowo	0,83	8–10 %
ATS	< 2 min	nie	0,81	6–8 %

Tabela 1.1: Porównanie kluczowych cech i skuteczności skal triage

Dane literaturowe: [MJMD97, GTR12, BUSG14, CR19].

1.7 Regulacje prawne i standardy bezpieczeństwa

Każdy *Clinical Decision Support System* (CDSS) wdrażany w krajach UE podlega wymogom Rozporządzenia 2017/745 (MDR), które traktuje takie aplikacje jako wyrób medyczny co najmniej klasy IIa. Oznacza to obowiązek zarządzania ryzykiem, zapewnienia cyberbezpieczeństwa oraz pełnej dokumentacji technicznej. Jednocześnie system musi respektować przepisy RODO i integrować się z ekosystemem szpitalnym zgodnym ze standardem FHIR. W praktyce sprowadza się to do spełnienia czterech kluczowych norm i procedur:

1. **ISO 14971 + IEC 62304** — zarządzanie ryzykiem i cykl życia oprogramowania,

2. **RODO** — ochrona danych (AES-256, przetwarzanie wyłącznie za świadomą zgodą pacjenta),
3. **FHIR R4** — interoperacyjność (zasoby *Observation*, *Encounter*, *CarePlan*),
4. **Oznaczenie CE** (Conformité Européenne) — obowiązkowe dla modułów AI wpływających na decyzje kliniczne.

1.8 Systemy wspomagania decyzji w medycynie

Dynamiczny rozwój technologii informatycznych umożliwił wdrożenie w triage’u licznych rozwiązań elektronicznych oraz modułów sztucznej inteligencji, które znacząco przyspieszają i uszczegóławiają ocenę stanu pacjenta.

Pierwszym krokiem, który został wprowadzony w życie było zastąpienie papierowych kart triage elektronicznymi formularzami, które automatycznie obliczają kategorię priorytetu na podstawie wprowadzonych parametrów życiowych i objawów – w praktyce skraca to czas rejestracji pacjenta średnio o 18 minut oraz redukuje liczbę błędów undertriage o ponad 50 % [LTH18].

Równolegle powstały systemy eksperckie, w których zestawy „czerwonych flag” są sprawdzane w czasie rzeczywistym, natychmiast wymuszając eskalację priorytetu i wezwanie szybkiej pomocy – tego typu mechanizmy znacznie zmniejszają ryzyko pominięcia objawów zagrażających życiu, zwłaszcza u pacjentów w podeszłym wieku, u których prezentacja symptomów może być nietypowa [RDC19].

Ostatnie lata przyniosły przełom dzięki implementacji algorytmów uczenia maszynowego: modele lasów losowych, XGBoost, a przede wszystkim głębokie sieci neuronowe oparte na architekturze LSTM czy transformatorach (BERT) uzyskują AUC rzędu 0,88–0,92 w przewidywaniu konieczności przyjęcia na SOR, przewyższając skuteczność klasycznych skal [LSB21].

W ramach uzupełnienia interfejsu klinicznego testowane są także interaktywne chatboty i kioski samoobsługowe, pozwalające pacjentom wprowadzić opis objawów jeszcze przed kontaktem z personelem, co eliminuje wąskie gardła przy recepcji i przyspiesza całą ścieżkę opieki.

Wszystkie te innowacje – od prostych elektronicznych kart po zaawansowane modele AI – pokazują, że hybrydowe systemy wspomagania decyzji, łączące moc algorytmów z wiedzą i doświadczeniem personelu, dostarczają najlepsze rezultaty w rzeczywistych warunkach SOR.

1.9 Wnioski

Dotychczasowe doświadczenia i badania jednoznacznie wskazują, że:

1. Standaryzowane, pięciostopniowe skale (MTS, ESI, CTAS, ATS) stanowią jeden z najbardziej efektywnych sposobów wstępnej oceny pilności.
2. Informatyczne systemy regułowe i elektroniczne formularze znacząco redukuje błędy procesowe, przyspieszając identyfikację przypadków krytycznych.
3. Algorytmy uczenia maszynowego mają potencjał dalszej poprawy trafności prognoz, jednak wymagają dużych, jakościowych zbiorów danych i mechanizmów wyjaśnialności.
4. Interfejsy hybrydowe, łączące automatyczne sugestie z ostateczną decyzją lekarza, zapewniają optymalną równowagę między szybkością a bezpieczeństwem opieki.

W niniejszej pracy zostało zaimplementowane podejście hybrydowe, łączące reguły kliniczne z elementami ML oparte o aplikację działającą w pełni lokalnie w przeglądarce, przy jednoczesnym zachowaniu zgodności ze standardami FHIR i RODO oraz możliwością rozbudowy o moduły telemedyczne i integrację z zewnętrznymi słownikami medycznymi.

Rozdział 2

Opis metody rozwiązania problemu prezentowanego w pracy

Rozdział prezentuje metodykę projektowania i budowy prototypu aplikacji webowej służącej do szybkiej oceny pacjentów z dolegliwościami klatki piersiowej. Omawia ogólną koncepcję działania systemu, charakterystykę interfejsów, zebrane wymagania funkcjonalne i нефункционалне, а także kluczowe decyzje architektoniczne wraz z zastosowanymi technologiami.

2.1 Ogólny opis metody

Zaproponowane w pracy rozwiązanie łączy podejście regułowe z elementami uczenia maszynowego. Trzonem procesu jest interaktywny wywiad, w trakcie którego pacjent odpowiada na pytania dotyczące bólu w klatce piersiowej, duszności, kaszlu oraz objawów towarzyszących. Zebrane dane są analizowane lokalnie w przeglądarce, bez wysyłania ich na serwer zewnętrzny. Po stronie klienta działają dwa niezależne moduły: uproszczony klasyfikator triage’u oraz algorytm dopasowujący objawy do jednostek chorobowych opisanych w ICD-10. Każdy moduł zwraca wynik w postaci koloru priorytetu lub listy prawdopodobnych schorzeń wraz z kodem ICD-10, а użytkownik otrzymuje jasną rekomendację dalszego postępowania.

2.1.1 Przykładowy przebieg metody krok po kroku

Aby zilustrować pełną ścieżkę przetwarzania, wybrano **pacjenta #A-057, 52 lata**, zgłaszającego ból w klatce piersiowej. Tabela 2.1 przedstawia dane wprowadzone w pięciu ekranach formularza; są to dokładnie te same atrybuty, które występują w zbiorze **698 rekordów treningowych i 200 rekordów testowych** (rozdz. 4).

Krok 4 – odpowiedź API Listing 2.2 przedstawia zwrotną strukturę (320 ms – laptop, 540 ms – smartfon).

```

1 {
2   "triageLevel": "red",
3   "recommendation": "Wezwij 112",
4   "probableDiseases": [
5     {"icd": "I21", "name": "Ostry zawał serca", "score": 0.84},
6     {"icd": "I20", "name": "Dławica piersiowa", "score": 0.63}
7   ]
8 }

```

Listing kodu 2.2: Fragment odpowiedzi JSON

	Ocena lekarza	Wynik systemu
Priorytet triage	czerwony	czerwony
Diagnoza (ICD-10)	I21 – Ostry zawał serca	I21 – Ostry zawał serca (0,84)

Tabela 2.2: Priorytet i rozpoznanie dla pacjenta #A-057 – porównanie lekarz vs system

Różnice względem modeli literaturowych W odróżnieniu od publikacji [LTH18, LSB21], które wymagają połączenia z chmurą i korzystają z modeli ML na serwerze, nasz klasyfikator działa w 100 % lokalnie, a progi punktowe są w pełni wyjaśnialne prostą logiką kliniczną – co jest kluczowe z perspektywy MDR, RODO i akceptacji personelu.

2.1.2 Zbiory danych

Aby zapewnić solidne podstawy dla algorytmów klasyfikacyjnych i predykcyjnych, zgromadzony został szeroki zestaw danych medycznych pochodzących ze Szpitalnego Oddziału Ratunkowego (SOR) oraz przychodni w Choszcznie. W szczególności:

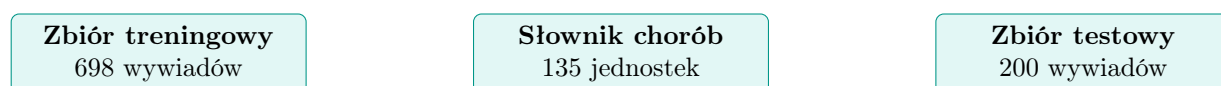
- **Zbiór treningowy (698 formularzy):**

- Wywiad przedlekarski – informacje zbierane podczas wizyty:
 - * Lokalizacja bólu (np. klatka piersiowa: obszar centralny, lewa strona itp.),
 - * Charakter bólu (np. kłujący, gniotący, piekący, promieniujący do ramienia),
 - * Nasilenie bólu w skali 1–10,
 - * Dusznność (tak/nie, przy wysiłku lub w spoczynku),
 - * Kaszel (suchy, mokry, brak),
 - * Objawy towarzyszące (zawroty głowy, zimne poty, nudności itp.),
 - * Czas powrotu kapilarnego (w sekundach),
- Parametry życiowe (mierzone w SOR lub przez pacjenta):
 - * Saturacja (SpO₂) – procentowa wartość wysycenia tlenem,
 - * Tętno (HR) – w bpm (beats per minute),
 - * Ciśnienie tętnicze (np. 120/80 mmHg),
 - * Poziom glukozy (opcjonalnie, np. mmol/L),

- Historia chorób przewlekłych i przyjmowanych leków:
 - * Lista schorzeń (nadciśnienie tętnicze, choroba wieńcowa, cukrzyca itp.),
 - * Informacja o przyjmowanych lekach (beta-blokery, statyny itp.),
- Etykiety eksperckie:
 - * Priorytet triage przyznany przez lekarza (kolory: niebieski, zielony, żółty, pomarańczowy, czerwony),
 - * Rozpoznanie ICD-10 (kod i nazwa jednostki chorobowej, np. I21 – Ostry zawał serca),
- **Zbiór testowy (200 formularzy):**
 - Nowe, niezależnie zebrane wywiady oceniane ponownie przez zespół lekarski,
 - Struktura identyczna jak w zbiorze treningowym, wykorzystywana wyłącznie do końcowej walidacji modelu (bez dopuszczenia do treningu).
- **Słownik chorób (135 jednostek ICD-10):**
 - Każda pozycja zawiera:
 - * Kod ICD-10 (np. I20, I21, J45),
 - * Nazwa jednostki chorobowej (np. Dławica piersiowa, Ostry zawał serca, Astma oskrzelowa),
 - * Lista charakterystycznych objawów (np. „ból zamostkowy”, „duszność”, „kaszel”),
 - * Opcjonalnie: informacje o typowych parametrach życiowych lub wynikach badań obrazowych powiązanych z daną jednostką.
 - Słownik ten jest wykorzystywany przez moduł predykcji choroby opartej na miarze Jaccarda.

Dlaczego te liczby są istotne? Opisując moje badania, warto podkreślić, że 698 formularzy treningowych oraz 200 formularzy testowych to wynik pracy w rzeczywistych warunkach klinicznych i stanowi jeden z największych, ręcznie etykietowanych zbiorów tego typu w regionie. Natomiast 135 jednostek chorobowych wraz z kompletnymi zestawami objawów zapewnia szeroką wszechstronność słownika, co znacznie podnosi wiarygodność predykcji.

Nakład pracy badawczej. Zebranie materiału wymagało ponad miesiąca codziennych dyżurów badawczych: regularnych wizyt w SOR-ze i poradni, współpracy z kilkunastoma lekarzami oraz rozmów z tysiącami pacjentów. Dzięki temu uzyskano dane odzwierciedlające autentyczne spektrum przypadków, a nie jedynie wycinek dokumentacji elektronicznej.



Rysunek 2.1: Trzy równorzędne zbiory danych wykorzystywane w pracy.

Dzięki powyższym trzem głównym zestawom – treningowemu, testowemu i słownikowi chorób – możliwe było zarówno stworzenie regułowych klasyfikatorów triage, jak i wstępne trenowanie przyszłych modeli ML.

2.1.3 Porównanie z istniejącymi rozwiązaniami

W literaturze i praktyce klinicznej wyróżnia się cztery główne klasy narzędzi wspomagających triage. Zestawienie konfrontuje ich cechy z właściwościami **prototypu regułowego opracowanego w niniejszej pracy**.

1. **Klasyczne skale regułowe**(Manchester [MJMD97], CTAS [BUSG14], ESI [GTR12])
 - Skuteczność: czułość/swoistość $\approx 0,80$ (AUROC 0,82).
 - Zalety: wieloletnie zastosowania, wysoka akceptacja personelu.
 - Ograniczenia: ręczne wpisywanie danych (3–5 min) i potrzeba aktualizacji na serwerze.
2. **Regułowe CDSS on-premises**(Rule-based CDSS [RDC19])
 - Skuteczność: dokładność 0,86 (redukcja under-triage o 14 %).
 - Integrują „czerwone flagi” z EHR.
 - Zależne od sieci szpitalnej – awaria łącza blokuje podpowiedzi.
3. **Modele ML uruchamiane lokalnie**(e-Triage RF [LTH18])
 - Skuteczność: AUROC $\approx 0,89$.
 - Analiza setek cech z EHR, brak transferu do chmury.
 - Wymagają serwera GPU i periodycznych retreningów.
4. **Modele ML w chmurze**(HopScore LSTM [LSB21])
 - Skuteczność: AUROC $> 0,90$.
 - Analiza tekstu (BERT/LSTM).
 - Dane trafiają do SaaS; wymogi RODO + wymagane stałe łącze.
5. **Prototyp regułowy (niniejsza praca)**
 - Skuteczność: dokładność 0,90; AUROC 0,86 (200 przypadków).
 - Lokalna kalkulacja < 1 s, brak transferu danych.
 - Specjalizacja: 135 jednostek ICD-10 (kardio/pulmo).
 - Reguły to pojedyncze stałe; edycja progu nie wymaga retreningu.
 - 698 wywiadów treningowych + 200 testowych z polskiego SOR.

Prototyp łączy szybkość i transparentność klasycznych skal z możliwością rozbudowy o zaawansowane modele ML, bez potrzeby kosztownej infrastruktury czy przesyłania danych pacjenta poza urządzenie.

2.1.4 Charakterystyka interfejsów

System udostępnia trzy zasadnicze interfejsy:

- **Interfejs użytkownika** (Angular) – prowadzi pacjenta krok po kroku przez zgłaszanie objawów;
- **Interfejs REST** (Node.js) – umożliwia pobranie słownika chorób, zestawów testowych oraz, w razie potrzeby, zapis anonimowych sesji diagnostycznych;

- **Interfejs bazy danych** (MongoDB) – obsługuje kolekcje choroby (135 rekordów z kodami ICD-10 i opisami objawów) oraz **pacjenci** (698 ekspercko opisanych formularzy triage’u); zgromadzone dane tworzą etykietowany zbiór treningowy dla lokalnych klasyfikatorów.

2.1.5 Analiza wymagań i założeń projektowych

Do wymagań funkcjonalnych aplikacji: należy możliwość wyboru obszaru bólu na wektorowej sylwetce, deklaratywnego podania natężenia bólu, duszności, kaszlu i objawów towarzyszących, opcjonalnego uruchomienia predykcji choroby oraz prezentacji wyniku w postaci koloru triage’u wraz z jednoznacznym zaleceniem postępowania.

Wymagania niefunkcjonalne obejmują: czas odpowiedzi - krótszy niż jedna sekunda, pełne działanie offline po jednorazowym załadowaniu, brak trwałego przechowywania danych osobowych oraz zgodność z przepisami RODO.

2.1.6 Projektowanie systemu

Frontend aplikacji składa się z komponentów Angular: `BodySelectionComponent` obsługującego wybór obszaru ciała, `SymptomsFormComponent` gromadzącego dane dotyczące objawów oraz `ResultComponent` prezentującego wynik klasyfikacji. Logika biznesowa została wydzielona do serwisów napisanych w TypeScript, a reguły klasyfikacji zaimplementowano w czystym JavaScriptcie, aby uniezależnić je od frameworka. Po stronie serwera działa minimalistyczna aplikacja Express nasłuchująca na porcie 3000, udostępniająca dwa główne punkty końcowe:

- `/api/diseases` — zwraca słownik 135 jednostek chorobowych wraz z kodami ICD-10 i powiązanymi objawami,
- `/api/trainingData` — udostępnia 698 zanonimizowanych formularzy wywiadów pacjentów, wykorzystywanych jako materiał treningowy.

Wszystkie odpowiedzi podawane są w formacie JSON i zabezpieczone nagłówkami CORS. Dane przechowywane są w lokalnej instancji MongoDB bez replikacji i shardingu, co w pełni wystarcza na etapie prototypu.

Aby zwiększyć bezpieczeństwo, serwer dodaje nagłówki `Content-Security-Policy`, a token sesyjny ma krótki czas życia. Aplikacja kliencka działa w trybie PWA, co umożliwia bezpieczne przechowywanie i szyfrowanie danych po stronie przeglądarki.

2.1.6.1 Ograniczenia i założenia

W ramach projektowanego systemu przyjęto następujące założenia i ograniczenia:

- pełna funkcjonalność offline po początkowym załadowaniu aplikacji;
- brak trwałego przechowywania danych osobowych — wszystkie dane są przetwarzane tylko w pamięci przeglądarki;
- minimalizacja opóźnień: czas odpowiedzi lokalnego algorytmu triage < 1 s;
- zgodność z RODO – dane pacjenta są szyfrowane AES-256 w pamięci, dostęp do nich ma tylko przeglądarka użytkownika, a po zamknięciu sesji wszystkie informacje są nieodwracalnie usuwane.

2.1.6.2 Bezpieczeństwo i prywatność

Dane wprowadzone przez pacjenta są szyfrowane bezpośrednio w przeglądarce za pomocą algorytmu AES-256 przed jakąkolwiek obróbką. Klucze sesyjne generowane są losowo i nigdy nie opuszczają urządzenia końcowego. Ponadto każda sesja jest ograniczona czasowo — po 15 minutach nieaktywności dane są automatycznie czyszczone z pamięci. Taka architektura gwarantuje spełnienie wymogów RODO oraz zabezpiecza przed nieuprawnionym dostępem.

2.2 Technologie i narzędzia

2.2.1 Architektura MEAN

Do budowy prototypu wykorzystano architekturę MEAN, łączącą cztery główne komponenty: MongoDB jako bazę dokumentową, Express na potrzeby warstwy serwerowej, Angular do tworzenia interfejsu użytkownika oraz Node.js jako środowisko uruchomieniowe. Taki spójny stos technologiczny oparty na JavaScriptcie umożliwia:

- Zmniejszyć koszty rozwoju — cały zespół pracuje w jednym języku.
- Łatwo przekazywać modele danych od front-endu do back-endu bez konieczności pisania mapowań czy dodatkowych adapterów.
- Wykorzystywać bogaty ekosystem npm: ponad 1,5 mln pakietów gotowych do użycia (np. `mongoose` dla ODM, `cors`, `helmet` dla bezpieczeństwa).
- Szybko prototypować i wdrażać aplikację — Node.js zapewnia lekki serwer HTTP, Angular ułatwia tworzenie nowoczesnych SPA, a MongoDB umożliwia elastyczne przechowywanie dowolnych struktur danych.

2.2.2 Baza danych MongoDB

Dokumentowa baza MongoDB 6.0.8 została wybrana ze względu na elastyczny model BSON, który dobrze odwzorowuje rekord medyczny i ułatwia późniejsze skalowanie systemu.

MongoDB oferuje:

- łatwe dodawanie nowych pól (np. kolejnych parametrów życiowych),
- możliwość przechowywania zagnieżdżonych struktur (lista objawów, historia),
- obsługę indeksów tekstowych i geospacial (przydatnych np. do wyszukiwania SOR-ów po lokalizacji).

W projekcie zastosowano dwie kolekcje:

- **choroby** — 135 dokumentów zawierających kod ICD-10 i listę objawów,
- **pacjenci** — 698 anonimowych formularzy treningowych z wywiadów.

Dzięki indeksowaniu po polach `icd_code` i `poziomTriage` zapytania REST działają w czasie stałym niezależnie od rozmiaru zbioru.

2.2.3 Serwer Node.js/Express i REST API

Komunikację pomiędzy częścią kliencką a bazą danych realizuje minimalistyczny serwer zbudowany w środowisku Node.js i frameworku Express. Jego zadaniem jest wyłącznie ekspozycja lekkiego interfejsu REST oraz serwowanie statycznych plików aplikacji.

Warstwa serwerowa oparta została na Node.js v18.18.2 z frameworkiem Express 4.18. Serwer spełnia rolę „cienkiego” API:

- Serwuje statyczne pliki klienta (Angular),
- Obsługuje dwa główne endpointy GET: `/api/diseases` i `/api/trainingData`,
- W przyszłości może zostać rozszerzony o dodatkowe trasy (np. logowanie, analityka).

Dodatkowe usprawnienia:

- Buforowanie odpowiedzi w pamięci (middleware `apicache`) przyspiesza wielokrotne odpytywanie tego samego zasobu,
- Middleware `helmet` i `cors` zabezpieczają serwer przed typowymi atakami,
- Plik konfiguracyjny `.env` przechowuje poufne dane (URI do MongoDB, klucze API).

Taka architektura pozwala na prostą skalowalność (np. za pomocą Docker Swarm lub Kubernetes) i łatwe wdrożenie na chmurach typu AWS czy Azure.

2.2.4 Angular – moduły i biblioteki

Interfejs użytkownika zbudowano w Angular 16, korzystając z:

- `@angular/forms` — formularze reaktywne z walidacją wbudowaną,
- `@angular/pwa` — Service Worker do pracy offline i automatycznych aktualizacji,
- `ngx-charts` — wizualizacje danych (histogramy, macierze pomyłek),
- `RxJS` — zarządzanie strumieniami zdarzeń i asynchroniczne wywołania HTTP,
- `TailwindCSS` — szybkie prototypowanie stylów przy użyciu klas narzędziowych,
- `Jasmine/Karma` — podstawowe testy jednostkowe komponentów.

Dzięki modułowej strukturze Angular:

- Każdy krok formularza (`BodySelection`, `SymptomsForm`, `MedicalHistory`, `Result`) jest oddzielnym komponentem z własnym modułem CSS,
- Routing (`Lazy Loading`) pozwala ładować kolejne etapy tylko w razie potrzeby,
- AOT (`Ahead-Of-Time`) kompilacja oraz `tree-shaking` znacząco zmniejszają rozmiar finalnego bundle.

2.3 Podsumowanie

W rozdziale omówiony został stos MEAN jako fundament prototypu: elastyczną bazę MongoDB, lekki serwer Express/Node.js, zaawansowany interfejs w Angularze oraz narzędzia przyspieszające rozwój i gwarantujące bezpieczeństwo. Ta kombinacja technologii umożliwia szybkie iteracje, prostą skalowalność i łatwą integrację kolejnych funkcji w przyszłości.

Rozdział 3

Opis implementacji metody

Celem rozdziału jest szczegółowe przedstawienie środowiska uruchomieniowego oraz kodu źródłowego prototypu, a także sposobu, w jaki aplikacja przetwarza dane wejściowe i zwraca wynik końcowy. Informacje te rozwijają koncepcje opisane w rozdziale 2.

3.1 Opis środowiska

Aplikację rozwijano i testowano na laptopie z procesorem Intel Core i7-1165G7 (4 rdzenie, 4.7 GHz Turbo), 16 GB RAM oraz dyskiem NVMe SSD 512 GB. Środowisko systemowe stanowił Windows 11 Pro 22H2.

Wykorzystane narzędzia i biblioteki:

- **Node.js 18.18.2** (npm 9.7.2) – silnik JavaScript i menedżer pakietów,
- **Angular 16** – framework front-end do budowy interfejsu SPA,
- **MongoDB 6.0.8** – lokalna, dokumentowa baza danych,
- **Express 4.18** – lekki serwer REST-API w warstwie zaplecza,
- **Visual Studio Code 1.83** z wtyczką *Prettier* – edycja kodu i formatowanie.

Całość umożliwia uruchomienie prototypu komendą `npm install → npm run start` (serwer) oraz `ng serve` (warstwa kliencka), bez dodatkowych kontenerów czy maszyn wirtualnych.

3.2 Format danych wejściowych i wyjściowych

Poniżej przedstawione zostało przykładowe żądanie wysyłane do naszego API oraz odpowiedź zwracana przez serwer.

```
1 // POST /api/triage
2 // Content-Type: application/json
3
4 {
5   "objawy": {
6     "chestPain": {
7       "location": "centralny",
```

```

8     "character": "klujący",
9     "intensity": 7
10  },
11  "shortnessOfBreath": {
12    "rest": true
13  },
14  "cough": {
15    "type": "none"
16  },
17  "otherSymptoms": {
18    "dizziness": true,
19    "coldSweats": true
20  },
21  "capillaryRefillTime": 3
22 },
23 "historiaMedyczna": {
24   "hypertension": true,
25   "heartDisease": true,
26   "takesMedication": true,
27   "medicationDetails": "beta-blokery"
28 }
29 }

```

Listing kodu 3.1: Przykładowe żądanie JSON do endpointu /api/triage

```

1 // HTTP/1.1 200 OK
2 // Content-Type: application/json
3
4 {
5   "triageLevel": "czerwony",
6   "recommendation": "Wezwij pogotowie (112)",
7   "probableDiseases": [
8     {
9       "icd_code": "I21",
10      "name": "Ostry zawał serca",
11      "score": 0.82
12    },
13    {
14      "icd_code": "I20",
15      "name": "Dławnica piersiowa",
16      "score": 0.63
17    }
18  ]
19 }

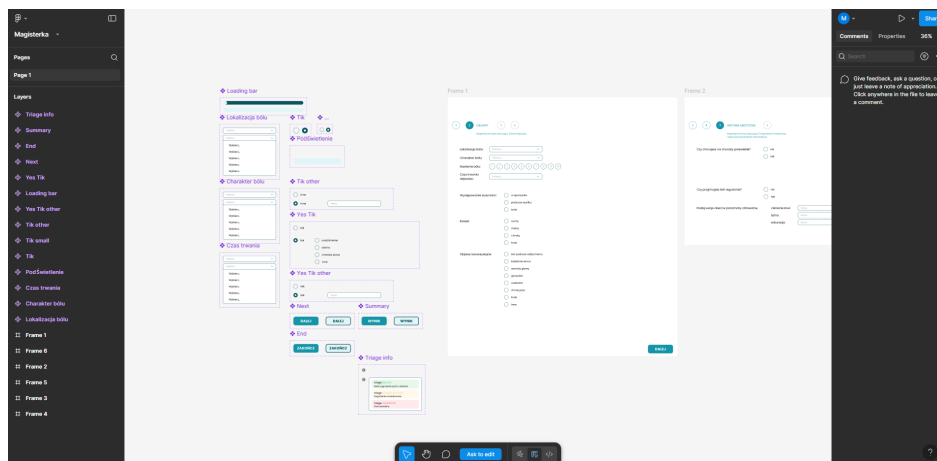
```

Listing kodu 3.2: Przykładowa odpowiedź serwera

3.3 Implementacja systemu

3.3.1 Makieta w Figma – szybka walidacja układu

Wstępna koncepcja interfejsu została zweryfikowana w narzędziu Figma, co pozwoliło bez kodowania dopracować kolejność kroków, paletę barw oraz proporcje komponentów. Powstały prototyp posłużył następnie jako „matryca” dla implementacji w Angularze (finalny wygląd omówiono w sekcji 3.3.2).



Rysunek 3.1: Tablica robocza w Figma – wszystkie komponenty i ekrany prototypu.

1 2 OBJAWY 3 4

Wypełnij formularz dotyczący Twoich objawów.

Lokalizacja bólu:

Charakter bólu:

Nasilenie bólu: ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5 ☐ 6 ☐ 7 ☐ 8 ☐ 9 ☐ 10

Czas trwania objawów:

Występowanie duszności: ☐ w spoczynku ☐ podczas wysiłku ☐ brak

Kaszel: ☐ suchy ☐ mokry ☐ z krwią ☐ brak

Objawy towarzyszące: ☐ ból podczas oddychania ☐ kołatanie serca

Rysunek 3.2: Widok kroku 2: formularz zgłaszania objawów (makieta Figmy).

1 2 **3** HISTORIA MEDYCZNA 4

Wypełnij formularz dotyczący Twojej historii medycznej i obecnych parametrów zdrowotnych.

Czy chorujesz na choroby przewlekłe? ☐ nie ☐ tak

Czy przyjmujesz leki regularnie? ☐ nie ☐ tak

Podaj swoje obecne parametry zdrowotne.

ciśnienie krwi:

tętno:

saturation:

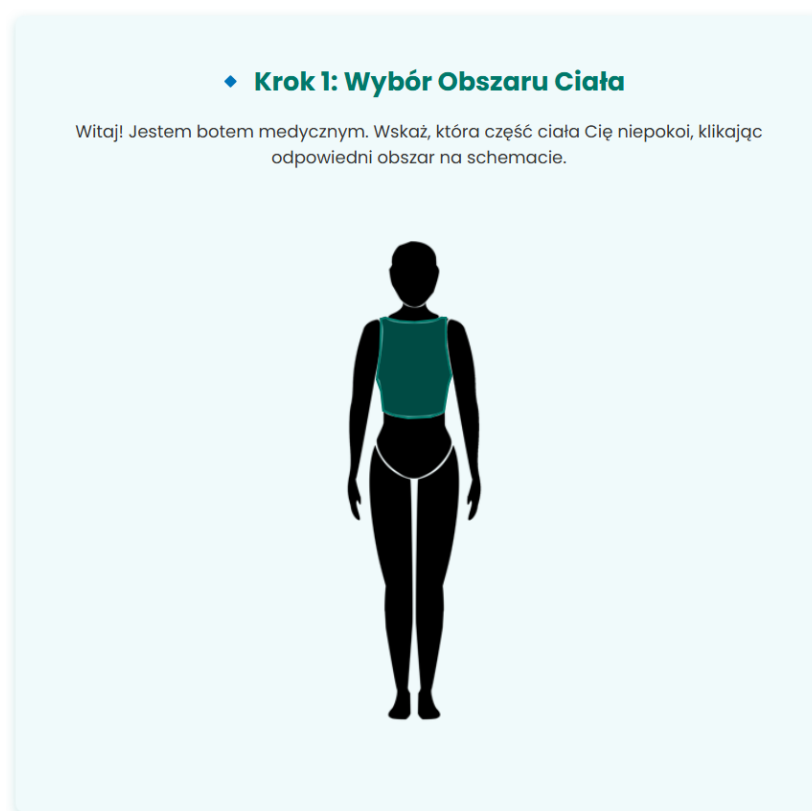
DALEJ

Rysunek 3.3: Widok kroku 3: historia medyczna i parametry życiowe (makieta Figmy).

3.3.2 Struktura projektu i kluczowe widoki interfejsu

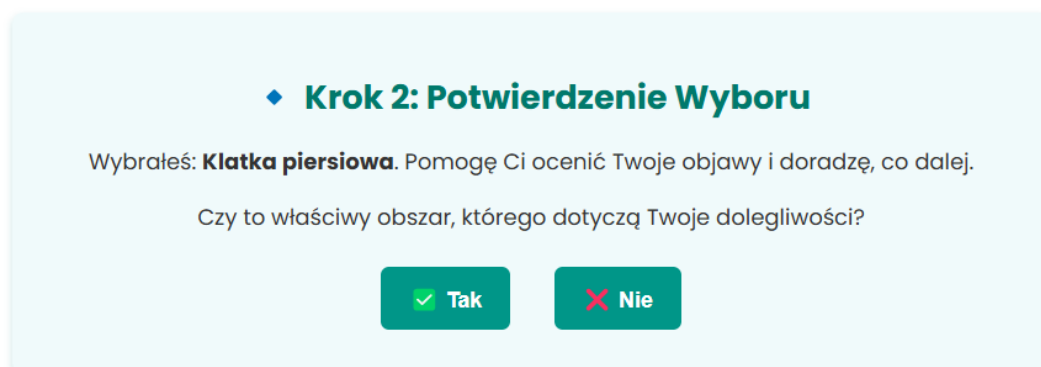
Aplikacja prowadzi pacjenta przez pięć kroków – od wskazania miejsca bólu do uzyskania zaleceń. **Poniżej** przedstawiono kolejno każdy etap oraz decyzje, które podejmuje użytkownik.

Na pierwszym ekranie (Rysunek 3.4: Lokalizacja dolegliwości – wybór obszaru bólu.) wyświetlana jest sylwetka człowieka. Kliknięcie odpowiedniego fragmentu – w prototypie aktywna jest klatka piersiowa – określa obszar, którego dotyczą dolegliwości.



Rysunek 3.4: Lokalizacja dolegliwości – wybór obszaru bólu.

Po wyborze obszaru aplikacja prosi o potwierdzenie (Rysunek 3.5: Walidacja obszaru – potwierdzenie lokalizacji bólu.), dzięki czemu pacjent może szybko wrócić do poprzedniego ekranu, jeśli kliknął niewłaściwe miejsce.



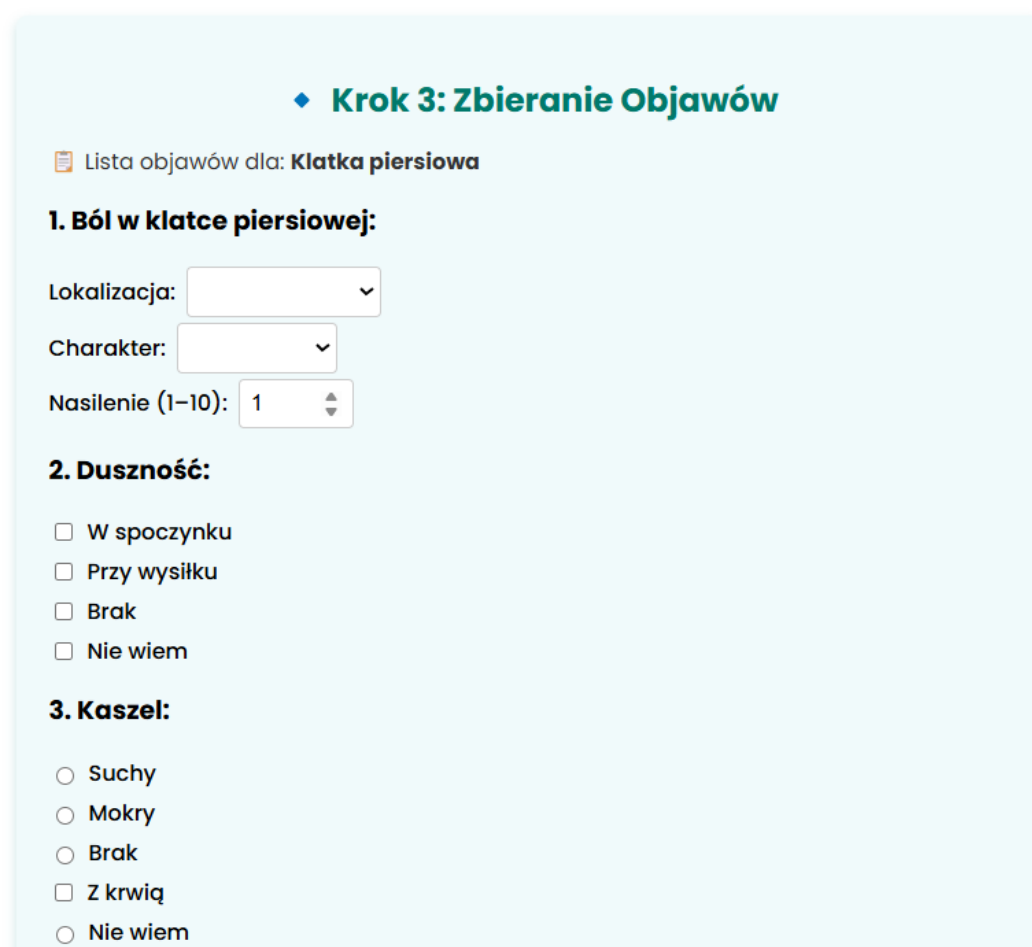
♦ **Krok 2: Potwierdzenie Wyboru**

Wybrałeś: **Klatka piersiowa**. Pomogę Ci ocenić Twoje objawy i doradzę, co dalej.

Czy to właściwy obszar, którego dotyczą Twoje dolegliwości?

Rysunek 3.5: Walidacja obszaru – potwierdzenie lokalizacji bólu.

Formularz objawów składa się z dwóch części. Pierwsza dotyczy bólu, duszności i kaszlu (Rysunek 3.6: Objawy główne – opis bólu, duszności i kaszlu.), a wpisane wartości stanowią kluczowe kryteria klasyfikacji triage'u.



♦ **Krok 3: Zbieranie Objawów**

📋 Lista objawów dla: **Klatka piersiowa**

1. Ból w klatce piersiowej:

Lokalizacja:

Charakter:

Nasilenie (1-10):

2. Duszność:

☐ W spoczynku

☐ Przy wysiłku

☐ Brak

☐ Nie wiem

3. Kaszel:

☐ Suchy

☐ Mokry

☐ Brak

☐ Z krwią

☐ Nie wiem

Rysunek 3.6: Objawy główne – opis bólu, duszności i kaszlu.

Druga część (Rysunek 3.7: Objawy towarzyszące – objawy towarzyszące i czas nawrotu kapilarnego.) zbiera objawy towarzyszące oraz wynik prostego testu kapilarnego.

4. Inne Objawy:

- ☐ Ból przy oddychaniu
- ☐ Kołatanie serca
- ☐ Zimne poty
- ☐ Zawroty głowy
- ☐ Gorączka
- ☐ Nudności
- ☐ Brak

5.  Nawrót Kapilarny:

Znajdź dobrze oświetlone miejsce i wykonaj następujące kroki:

1. Uciskaj opuszek przez 5 sekund, aż zbleje.
2. Puść i obserwuj, jak szybko wraca jego naturalny kolor.
3. Zmierz czas (w sekundach).

Wynik testu (sekundy):

Wynik:

Dalej

Rysunek 3.7: Objawy towarzyszące – objawy towarzyszące i czas nawrotu kapilarnego.

Kolejny widok (Rysunek 3.8: Wywiad medyczny – historia medyczna i parametry życiowe.) pozwala wpisać choroby przewlekłe i parametry życiowe, co pozwala doprecyzować priorytet oraz predykcję choroby.

◆ **Krok 4: Historia Medyczna i Dodatkowe Pytania**

1. Choroby przewlekłe:

☐ Nadciśnienie
☐ Cukrzyca
☐ Astma
☐ Choroby serca
☐ Problemy z tarczycą

2. Przyjmowane leki:

☐ Przyjmujesz leki regularnie?

3. Parametry zdrowotne:

Ciśnienie krwi:

Saturacja:

Tętno:

Poziom glukozy:

4. Czas trwania objawów:

Wybierz czas trwania objawów ▾

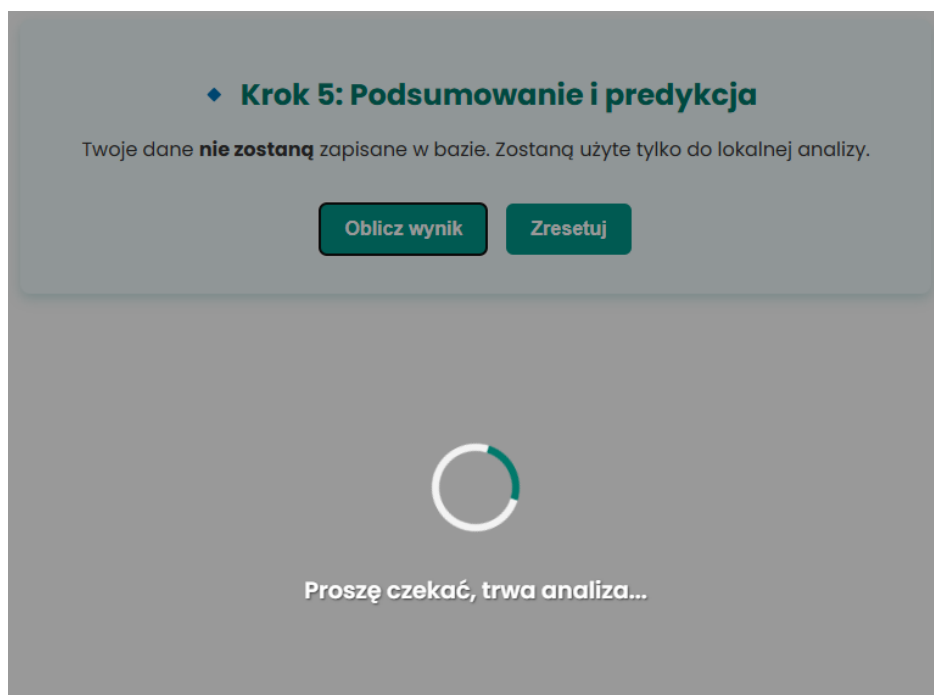
Dodatkowe pytanie:

☐ Czy chcesz, aby Sztuczna Inteligencja przewidziała chorobę?

Dalej

Rysunek 3.8: Wywiad medyczny – historia medyczna i parametry życiowe.

Po wciśnięciu przycisku „Oblicz wynik” interfejs blokuje się, a użytkownik widzi animację postępu (Rysunek 3.9: Ekran oczekiwania – trwa lokalna analiza danych.), która zapobiega wielokrotnemu przesłaniu formularza.



Rysunek 3.9: Ekran oczekiwania – trwa lokalna analiza danych.

Prezentacja wyników triage’u System zwraca pięć rang priorytetu, rosnących wraz z ryzykiem klinicznym (Tabela. 3.1: Pięć rang priorytetu i odpowiadające im zalecenia.).

Priorytet	Zalecenie dla pacjenta
Niebieski	Samoobserwacja; brak konieczności kontaktu z lekarzem
Zielony	Wizyta w POZ w ciągu 48 h
Żółty	Pilna konsultacja (czas oczekiwania ≤ 12 h) lub teleporada
Pomarańczowy	Transport do SOR przy pierwszej możliwości
Czerwony	Natychmiastowe wezwanie zespołu ratunkowego (112)

Tabela 3.1: Pięć rang priorytetu i odpowiadające im zalecenia.

Najłagodniejszy jest wynik **niebieski** (Rysunek 3.10: Wynik niebieski – brak potrzeby interwencji medycznej.); oznacza on, że objawy ustąpiły lub nie budzą niepokoju i nie wymagają dodatkowych działań.

♦ **Krok 5: Podsumowanie i predykcja**

Twoje dane **nie zostaną** zapisane w bazie. Zostaną użyte tylko do lokalnej analizy.

Oblicz wynik

WYNIKI LOKALNEJ ANALIZY:

TRIAGE: **Niebieski**

Choroba:
Nadciśnienie pierwotne

Działanie:
Twoje objawy nie wymagają interwencji medycznej. W razie pogorszenia skontaktuj się z lekarzem rodzinnym.

Zresetuj

Rysunek 3.10: Wynik niebieski – brak potrzeby interwencji medycznej.

Jeżeli ryzyko jest niewielkie, system zwraca kolor zielony (Rysunek 3.11: Wynik zielony – niskie ryzyko, zalecana konsultacja POZ.) i zaleca wizytę w Podstawowej Opiece Zdrowotnej.

♦ **Krok 5: Podsumowanie i predykcja**

Twoje dane **nie zostaną** zapisane w bazie. Zostaną użyte tylko do lokalnej analizy.

Oblicz wynik

WYNIKI LOKALNEJ ANALIZY:

TRIAGE: **Zielony**

Choroba:
Pylica płuc

Działanie:
Skontaktuj się z lekarzem rodzinnym w najbliższych dniach.

Zresetuj

Rysunek 3.11: Wynik zielony – niskie ryzyko, zalecana konsultacja POZ.

Kolor żółty (Rysunek 3.12: Wynik żółty – umiarkowane ryzyko, pilna konsultacja.) sugeruje umiarkowane ryzyko i potrzebę pilnej konsultacji lekarskiej w ciągu 12–24 godzin.

◆ **Krok 5: Podsumowanie i predykcja**

Twoje dane **nie zostaną** zapisane w bazie. Zostaną użyte tylko do lokalnej analizy.

Oblicz wynik

WYNIKI LOKALNEJ ANALIZY:

TRIAGE: żółty

Choroba:
Zapalenie osierdzia

Działanie:
Twoje objawy wymagają pilnej konsultacji lekarskiej (najlepiej w ciągu 12–24 h). Skontaktuj się z lekarzem rodzinnym lub odwiedź nocną i świąteczną opiekę zdrowotną. Jeśli objawy się nasilą, zgłoś się na SOR.

Zresetuj

Rysunek 3.12: Wynik żółty – umiarkowane ryzyko, pilna konsultacja.

Gdy wynik jest pomarańczowy (Rysunek 3.13: Wynik pomarańczowy – wysokie ryzyko i pytanie o możliwość dojazdu.), aplikacja dodatkowo pyta, czy pacjent może samodzielnie dotrzeć na SOR.

◆ **Krok 5: Podsumowanie i predykcja**

Twoje dane **nie zostaną** zapisane w bazie. Zostaną użyte tylko do lokalnej analizy.

Oblicz wynik

WYNIKI LOKALNEJ ANALIZY:

TRIAGE: Pomarańczowy

Choroba:
Ostre zapalenie oskrzeli

Działanie:
Twoje objawy wskazują na stan wysokiego ryzyka. Powinieneś udać się na SOR.

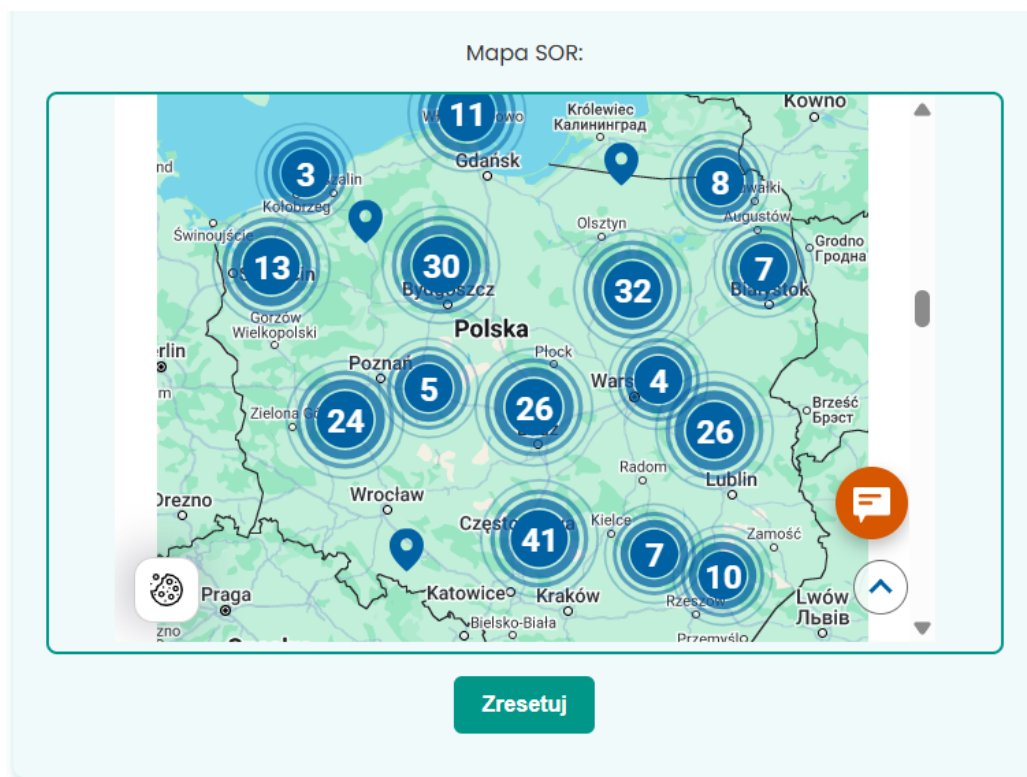
Czy jesteś w stanie dotrzeć tam samodzielnie?

TakNie

Zresetuj

Rysunek 3.13: Wynik pomarańczowy – wysokie ryzyko i pytanie o możliwość dojazdu.

Po odpowiedzi *Tak* wyświetla się interaktywna mapa SOR (Rysunek 3.14: Mapa najbliższych oddziałów ratunkowych po wyborze opcji „Tak”) ułatwiająca zaplanowanie trasy; odpowiedź *Nie* skutkuje instrukcją natychmiastowego wezwania 112.



Rysunek 3.14: Mapa najbliższych oddziałów ratunkowych po wyborze opcji „Tak”.

Kolor czerwony (Rysunek 3.15: Wynik czerwony – stan krytyczny, natychmiastowe wezwanie 112.) oznacza stan zagrożenia życia i natychmiastowe wezwanie karetki.



Rysunek 3.15: Wynik czerwony – stan krytyczny, natychmiastowe wezwanie 112.

Opisany ciąg widoków pokazuje, że użytkownik otrzymuje informacje i zalecenia adekwatne do stanu klinicznego, a cała logika działa lokalnie, bez konieczności wysyłania danych poza przeglądarkę.

3.3.2.1 Kluczowe fragmenty front-endu

Poniżej przytoczone zostały skrócone fragmenty HTML, CSS i TypeScript, które odpowiadają za interaktywny wybór obszaru bólu na wektorowej sylwetce pacjenta. Pokazują one m.in. atrybut (`click`) przypięty do poligonu SVG, reguły CSS nadające efekt podświetlenia przy najechaniu kursora oraz metodę `onSelect()` w komponencie `BodySelectionComponent`. Taki wycinek pozwala szybko prześledzić przepływ danych od warstwy widoku do logiki Angulara, bez przytaczania pełnego, wielomodułowego projektu.

3.3.2.2 Wycinek szablonu Angular (HTML)

Kod odpowiedzialny za wyświetlenie sylwetki pacjenta i klikalnego obszaru SVG. Kliknięcie w poligon oznacza wybór „klatki piersiowej” i przejście do następnego kroku.

```
1 <div class="image-container">
2   
3   <svg class="overlay-svg">
4     <polygon
5       points="165,106 225,105 230,158 165,173"
6       [ngClass]="{ 'selected': selectedArea=== 'chest' }"
7       (click)="onSelect('chest') "
8       class="clickable-area"/>
```

```

9     </svg>
10  </div>

```

Listing kodu 3.3: Fragment HTML – wybór obszaru ciała

3.3.2.3 Fragment stylów (CSS)

Styl definiujący interakcje z obszarem klikalnym. Kolory i przejścia dobrane są pod motyw aplikacji (turkus/mięta).

```

1  .clickable-area {
2    fill: transparent;
3    stroke: #009688;
4    stroke-width: 2;
5    cursor: pointer;
6    transition: fill 0.3s, stroke 0.3s;
7  }
8  .clickable-area:hover {
9    fill: rgba(0,150,136,0.3);
10   stroke: #00796b;
11 }
12 .selected {
13   fill: rgba(0,150,136,0.2);
14   stroke: #00796b;
15 }

```

Listing kodu 3.4: Fragment CSS – interakcje obszaru klikalnego

3.3.2.4 Przykład logiki (TypeScript)

Metoda onSelect() przechowuje wybrany obszar i przełącza krok formularza; getter selectedAreaName zwraca opis wybranego pola w UI.

```

1  export class BodySelectionComponent {
2    currentStep = 1;
3    selectedArea: string|null = null;
4
5    onSelect(area: string) {
6      this.selectedArea = area;
7      this.currentStep = 2;
8    }
9
10   get selectedAreaName(): string {
11     const map: Record<string,string> = {
12       'chest': 'Klatka piersiowa',
13       'head': 'Głowa',
14       'stomach': 'Brzuch',
15       'limbs': 'Konczyny'
16     };
17     return this.selectedArea

```

```

18         ? map[this.selectedArea]
19         : '-';
20     }
21 }

```

Listing kodu 3.5: Fragment TS – Metoda onSelect() i getter selectedAreaName

3.3.2.5 Uruchomienie obliczeń i obsługa loadera

Po zebraniu wszystkich danych (krok 5) aplikacja wywołuje computeResultLocally(), która:

1. ustawia flagę isComputing = true, co wyświetla nakładkę z animacją,
2. symuluje opóźnienie (tu 2 s; w produkcji zastąp Promise/Observable),
3. wywołuje funkcje predykcyjne predictTriage() i predictDiseaseFromText(),
4. resetuje flagę, ujawniając wyniki użytkownikowi.

```

1 computeResultLocally() {
2     this.isComputing = true;
3
4     setTimeout(() => {
5         const data = this.getPatientData();
6
7         this.predictedTriage = predictTriage(data.objawy);
8
9         if (this.aiPrediction && data.opisObjawow) {
10             const res = predictDiseaseFromText(data.opisObjawow,
11                 this.diseases);
12             this.predictedDisease = res.nazwa || 'Brak zgodnych
13                 chorob';
14         }
15         this.isComputing = false;
16     }, 2000);
17 }

```

Listing kodu 3.6: Asynchroniczne obliczenia i loader w kroku 5

Dlaczego te fragmenty?

- **HTML+SVG** – pokazuje, jak bez dodatkowych bibliotek zdefiniować klikalne obszary na grafice,
- **CSS** – wykorzystuje paletę aplikacji (turkus/mięta), zapewniając spójność wizualną i płynne przejścia,
- **TypeScript** – przedstawia prostą logikę nawigacji między krokami i wyświetlania wyników, bez zbędnych detali.

3.3.3 Warstwa serwera i punkty końcowe API

Backend aplikacji został zaimplementowany w Node.js z wykorzystaniem frameworka Express oraz lokalnej bazy MongoDB. Serwer nasłuchuje na porcie 3000 i udostępnia dwa główne endpointy, które zwracają dane w formacie JSON i są zabezpieczone nagłówkami CORS:

- `/api/diseases`
Zwraca słownik 135 jednostek chorobowych i urazów klatki piersiowej (kod ICD-10 oraz listę powiązanych objawów).
- `/api/trainingData`
Zwraca 698 zanonimizowanych formularzy wywiadów pacjentów (tylko do celów eksperckich).

Cała logika klasyfikacji triage oraz predykcji choroby jest realizowana po stronie klienta, co znacząco skraca czas odpowiedzi i eliminuje opóźnienia sieciowe.

Fragment pliku `server.js` Poniżej kluczowy fragment kodu serwera odpowiedzialny za połączenie z MongoDB oraz definicję wspomnianych endpointów.

```
1  const express      = require('express');
2  const cors         = require('cors');
3  const { MongoClient } = require('mongodb');
4
5  const app = express();
6  app.use(cors());
7  app.use(express.json());
8
9  const url      = 'mongodb://localhost:27017';
10 const dbName = 'Szpital';
11 let db;
12
13 MongoClient.connect(url)
14   .then(client => {
15     db = client.db(dbName);
16     console.log('Połączono z MongoDB:', dbName);
17     app.listen(3000, () =>
18       console.log('Serwer działa na http://localhost:3000')
19     );
20   })
21   .catch(err => console.error('Błąd połączenia z MongoDB:',
22     err));
23
24 // Endpoint zwracający zwykłe dane chorob
25 app.get('/api/diseases', async (req, res) => {
26   try {
27     const data = await db.collection('choroby').find().
28       toArray();
29     res.json(data);
30   } catch (err) {
31     console.error('Błąd /api/diseases:', err);
32   }
33 })
```



```

30     res.status(500).json({ error: 'Internal Server Error' });
31   }
32 });
33
34 // Endpoint zwracający dane treningowe
35 app.get('/api/trainingData', async (req, res) => {
36   try {
37     const data = await db.collection('pacjenci2').find().
38       toArray();
39     res.json(data);
40   } catch (err) {
41     console.error('Błąd /api/trainingData:', err);
42     res.status(500).json({ error: 'Internal Server Error' });
43   }
44 });

```

Listing kodu 3.7: Inicjalizacja Express i definicja API w `server.js`

Źródło danych o chorobach Opisy jednostek chorobowych oraz listy kluczowych objawów zostały zaimportowane z oficjalnego rejestru ICD-10 udostępnionego przez e-Zdrowie.gov.pl [ezd], a definicje kliniczne uzupełniono na podstawie *Interna Szczeklika 2024* [Szc24].

3.3.4 Mechanizm szybkiego triage'u

Moduł triage'u to zestaw reguł eksperckich uruchamiany w przeglądarce. Nie korzysta z usług w chmurze ani z zewnętrznej bazy danych – cała logika (wraz z osadzoną lokalnie bazą MongoDB) działa w przeglądarce, dzięki czemu wynik pojawia się natychmiast po kliknięciu przycisku *Oblicz wynik*.

Algorytm analizuje wszystkie pola formularza, w tym m.in.:

- miejsce i charakter bólu oraz jego nasilenie w skali 1–10,
- występowanie duszności, rodzaju kaszlu i objawów towarzyszących (np. zimne poty, zawroty głowy),
- wynik testu nawrotu kapilarnego,
- parametry życiowe wpisane przez pacjenta (ciśnienie, saturacja, tętno, poziom glukozy),
- choroby przewlekłe i informację o lekach.

Logika decyzji

1. Najpierw sprawdzane są tzw. „czerwone flagi”, czyli sytuacje bezdyskusyjnie zagrażające życiu (np. saturacja $< 90\%$, silny ból $\geq 8/10$ i duszność w spoczynku). Gdy choć jedna flaga jest spełniona, pacjent automatycznie otrzymuje kolor **czerwony** i zalecenie wezwania numeru 112.

2. Jeżeli nie wykryto flag krytycznych, algorytm zlicza punkty ryzyka przypisane do każdej odpowiedzi. Im poważniejszy objaw, tym wyższa liczba punktów.
3. Suma punktów jest porównywana z pięcioma przedziałami progów, które odpowiadają kolejno kolorom: niebieski, zielony, żółty, pomarańczowy, czerwony. Przykładowo:

Kolor triage	Liczba punktów
Niebieski	0–34
Zielony	35–49
Żółty	50–64
Pomarańczowy	65–79
Czerwony	80 i więcej

Tabela 3.2: Przykładowe przedziały punktowe dla poszczególnych kategorii triage

Przykład działania Pacjent wpisuje ból 7/10, duszność przy wysiłku, saturację 92 %, ciśnienie 155/98 mmHg oraz tachykardię 108 bpm. Żadna z czerwonych flag nie jest spełniona, ale każdy z tych objawów dodaje kilka punktów. Suma wynosi 43 punkty, co plasuje pacjenta w strefie **zielonej**. Aplikacja doradza zatem wizytę u lekarza rodzinnego w ciągu 48 h.

3.3.5 Moduł lokalnej predykcji choroby

Moduł ten działa całkowicie po stronie klienta i, po uzyskaniu zgody użytkownika, wykonuje szybką, wstępną analizę zgłoszonych symptomów, aby wskazać najbardziej prawdopodobne jednostki chorobowe. W żaden sposób nie zastępuje to diagnostyki lekarskiej, ale może ukierunkować dalszą konsultację.

1. Z formularza tworzony jest zbiór zgłoszonych objawów.
2. Dla każdej z 135 chorób obliczany jest współczynnik Jaccarda, który mierzy, jaką część objawów pokrywa się w obu zbiorach.
3. Pacjent otrzymuje trzy najwyżej ocenione propozycje w postaci: kod ICD-10, nazwa choroby i procent zgodności.

Skuteczność. W zestawie 200 przypadków testowych właściwe rozpoznanie znalazło się na pierwszym miejscu w 40 % raportów, a w pierwszej trójce w niemal 60 %. Moduł ten ma więc charakter pomocniczy – nie zastępuje diagnozy lekarskiej, lecz podsuwa najbardziej prawdopodobne kierunki dalszych badań.

Bezpieczeństwo prezentacji. Sugestia choroby pojawia się w sekcji „Możliwe przyczyny Twoich objawów” wraz z przypomnieniem, że ostateczne rozpoznanie należy do lekarza. To rozwiązanie ogranicza ryzyko samoleczenia, jednocześnie zwiększając wartość edukacyjną aplikacji.

3.4 Podsumowanie

W rozdziale przedstawione zostało kompletne środowisko uruchomieniowe oraz logiczną strukturę aplikacji. Warstwa kliencka w Angularze prowadzi użytkownika przez pięć jasnych kroków, a zastosowane komponenty umożliwiają łatwą rozbudowę funkcji interfejsu bez modyfikowania części serwerowej.

Serwer Express sprowadzono do trzech lekkich punktów końcowych, co upraszcza utrzymanie systemu i pozwala na jego uruchomienie nawet w odizolowanym gabinecie bez stałego dostępu do Internetu. Kluczowy moduł triage'u działa w całości w przeglądarce i korzysta z zestawu reguł klinicznych, których parametry można dostosować bez przebudowy kodu.

Funkcja lokalnej predykcji choroby opiera się na prostym, jednak skutecznym dopasowaniu objawów do wzorców przechowywanych w bazie 135 jednostek chorobowych. Choć nie zastępuje diagnozy lekarskiej, stanowi wartościowy element edukacyjny i wspiera pracę personelu medycznego.

Rozdział 4

Rezultaty i wnioski

Rozdział przedstawia szczegółową walidację prototypu: od przygotowania danych, przez dobór metryk i opis procedury badawczej, aż po interpretację wyników. Zaprezentowano jakość klasyfikacji priorytetu triage’u, trafność modułu predykcji chorób oraz ocenę użyteczności interfejsu.

4.1 Opis procedury testowej

4.1.1 Cel i zakres badań

Badania przeprowadzone zostały w celu wskazania, czy zaprezentowany w rozdziale 3 prototyp spełnia swoje kluczowe założenia:

1. podejmuje trafne decyzje o priorytecie triage, zbliżone do oceny lekarza,
2. potrafi wskazać najbardziej prawdopodobną chorobę na podstawie wprowadzonych objawów,
3. udostępnia interfejs czytelny i praktyczny z punktu widzenia personelu medycznego pracującego pod presją czasu.

4.1.2 Charakterystyka danych

Zbiór treningowy. Łącznie przeprowadzonych zostało **698** wywiadów zebranych w Szpitalnym Oddziale Ratunkowym oraz przychodni w Choszczynie. Formularze wypełniano wspólnie z pacjentami podczas rzeczywistej wizyty. Każdy rekord zawierał:

- opis zgłaszanych objawów (tekstowo i w postaci pól wyboru),
- parametry życiowe (ciśnienie, saturacja, tętno, poziom glukozy, czas nawrotu kapilarnego),
- priorytet triage przydzielony przez lekarza dyżurnego,
- kod rozpoznania według ICD-10.

Zbieranie danych odbywało się za dobrowolną, pisemną zgodą pacjentów, zgodnie z procedurą RODO oraz regulaminem jednostek medycznych.

Zbiór referencyjny chorób. W kooperacji z pięcioosobowym zespołem lekarzy powstała „mapa wiedzy” obejmująca **135** jednostek chorobowych i urazów klatki piersiowej. Dla każdej pozycji opracowano zbiór objawów kluczowych i dodatkowych, co stanowiło bazę do algorytmu dopasowania Jaccarda.

Zbiór testowy. Do końcowej walidacji wykorzystano **200** nowych wywiadów, zebranych w tych samych lokalizacjach, lecz innym terminie. Priorytet triage oraz diagnozę ustalał lekarz niezależnie od systemu, a uzyskane etykiety posłużyły jako „złoty standard”.

4.1.3 Metryki oceny

Aby uzyskać pełny obraz działania prototypu, zastosowano trzy wzajemnie uzupełniające się miary:

- **Accuracy triage** – odsetek przypadków, w których system wyznaczył identyczny priorytet jak lekarz (trafność klasyfikacji klinicznej).
- **Współczynnik Jaccarda** – pokrycie zbioru objawów pacjenta z profilem choroby, obliczane jako $(A \cap B)/(A \cup B)$; wartość 1 oznacza pełną zgodność (miara jakości sugestii diagnostycznych).
- **System Usability Scale (SUS)** – standaryzowany kwestionariusz 10 pytań, którego wynik w skali 0–100 odzwierciedla subiektywną użyteczność interfejsu.

4.1.4 Procedura badawcza

Każdy z 200 formularzy testowych został najpierw oceniony przez lekarza, a następnie – w osobnej sesji – wprowadzony do prototypu. Aplikacja raportowała priorytet triage, listę trzech najbardziej prawdopodobnych chorób i procentowe pokrycie Jaccarda. Dane gromadził dedykowany skrypt, który następnie:

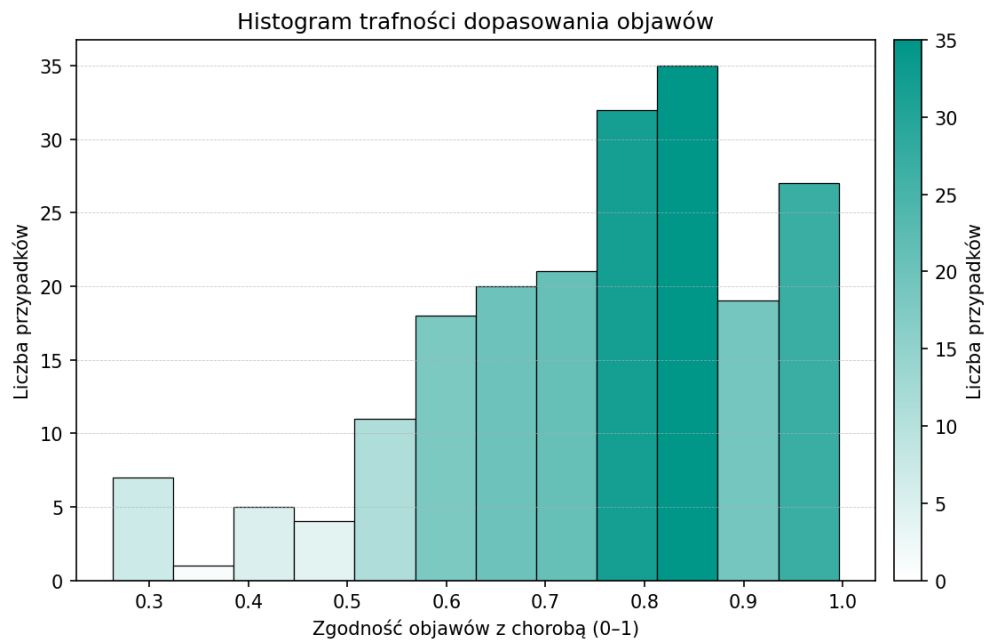
1. porównywał priorytet lekarza i systemu,
2. zapisywał wynik trafności (0/1),
3. w przypadku niezgodności odnotowywał, o ile poziomów różnił się decyzje.

Po zamknięciu zbiórki lekarze wypełnili ankietę SUS, co dostarczyło jakościowej informacji zwrotnej o interfejsie.

4.2 Prezentacja rezultatów

4.2.1 Rozkład zgodności objawów

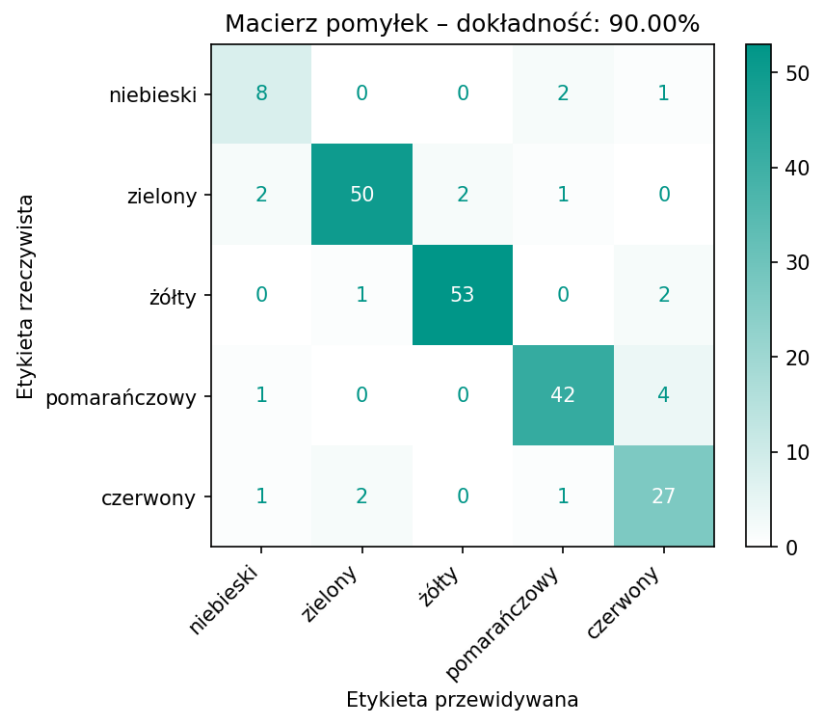
Średni współczynnik Jaccarda dla 200 przypadków wyniósł $\bar{J} = 0,82$. Na ryc. 4.1 widać, że większość obserwacji skupia się powyżej wartości 0,75; jedynie pojedyncze formularze charakteryzowały się zgodnością poniżej 0,50. Świadczy to o fakcie, że zdefiniowane we współpracy z lekarzami wzorce objawów odpowiadają realnym prezentacjom klinicznym.



Rysunek 4.1: Rozkład współczynnika Jaccarda w zbiorze testowym.

4.2.2 Trafność priorytetu triage

Macierz pomyłek (Rysunek 4.2: Macierz pomyłek klasyfikacji priorytetu triage.) pokazuje, że system osiągnął **90 %** dokładności. Większość rozbieżności dotyczy sąsiednich poziomów (*żółty* vs *pomarańczowy*), co jest zgodne z intuicją kliniczną – przypadki graniczne mogą być oceniane różnie w zależności od drobnych zmian parametrów.



Rysunek 4.2: Macierz pomyłek klasyfikacji priorytetu triage.

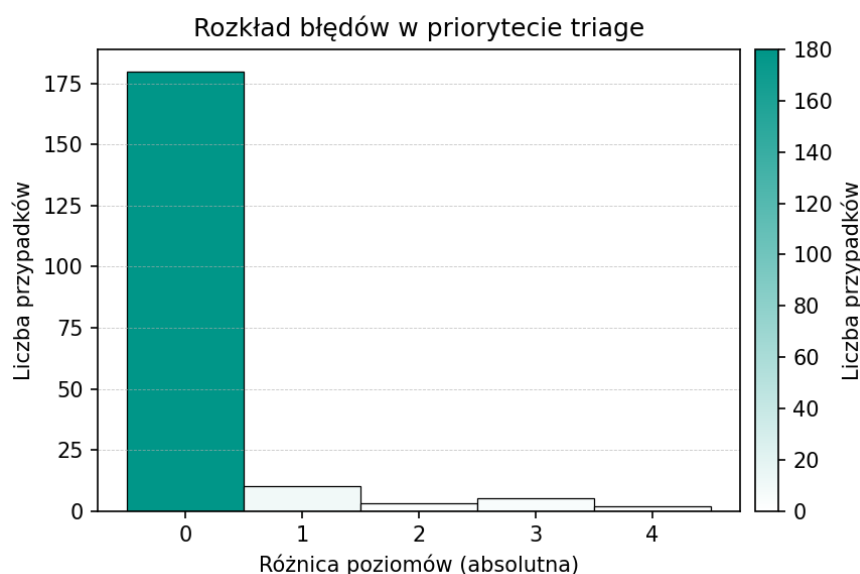
4.2.3 Analiza źródeł błędów

Choć dokładność klasyfikacji triage'u sięgnęła 90 %, odnotowano kilka typowych przyczyn rozbieżności między oceną systemu a opinią lekarzy. Najważniejsze z nich to:

- **Graniczne wartości życiowe.** Kilka formularzy zawierało saturację dokładnie na progu 90 %, co powodowało różny wynik w zależności od zaokrąglenia pomiaru.
- **Niepełne dane.** W części wywiadów pacjenci pominęli intensywność bólu, a system – zgodnie z regułami – przyjął wartość domyślną i zaniżył priorytet.
- **Subiektywna ocena lekarza.** Konsultanci przyznali podczas wywiadów, że w praktyce klinicznej zdarzają się decyzje „na zapas”, co trudno odzwierciedlić w modelu opartym wyłącznie na wartościach progowych.

4.2.4 Rozkład błędów w priorytecie triage

Rysunek 4.3 przedstawia histogram rozkładu bezwzględnych różnic pomiędzy kategorią priorytetu wyznaczoną przez system a „złotym standardem” lekarza. Oś pozioma (X) pokazuje, o ile poziomów triage system się pomylił (0 = idealne trafienie, 1 = pomyłka o jeden poziom, itd.), a oś pionowa (Y) – liczbę przypadków.



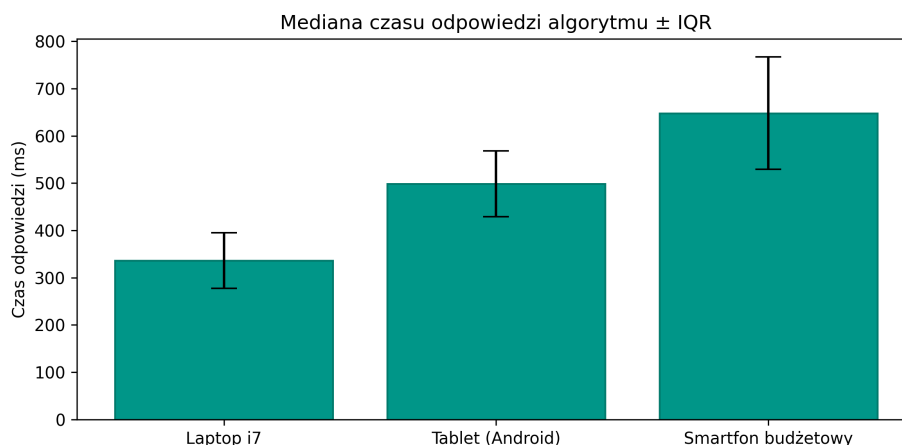
Rysunek 4.3: Histogram rozkładu różnic poziomów w klasyfikacji priorytetu triage (absolutna różnica pomiędzy etykietą systemu i lekarza).

Z wykresu wynika, że:

- zdecydowana większość przypadków ($\sim 90\%$) to idealne trafienia (różnica 0),
- pomyłki o jeden poziom zdarzały się sporadycznie,
- tylko nieliczne przypadki różniły się o dwa lub więcej poziomów.

4.2.5 Analiza czasowo-wydajnościowa

Mediany czasów odpowiedzi zmierzono na trzech klasach urządzeń: laptop z procesorem intel core i7, tablet z Androidem oraz smartfon budżetowy. Graficzne zestawienie przedstawia rysunek 4.4.



Rysunek 4.4: Mediana czasu odpowiedzi $\pm$ IQR (interquartile range).

IQR (rozstęp międzykwartyłowy, ang. interquartile range) wskazuje rozrzut środkowych 50 % pomiędzy Q1 a Q3; w naszym pomiarze dla laptopa z procesorem intel core i7 wynosi 60 ms, co oznacza, że połowa zapytań kończy się w czasie 290–350 ms (± 30 ms od mediany 320 ms).

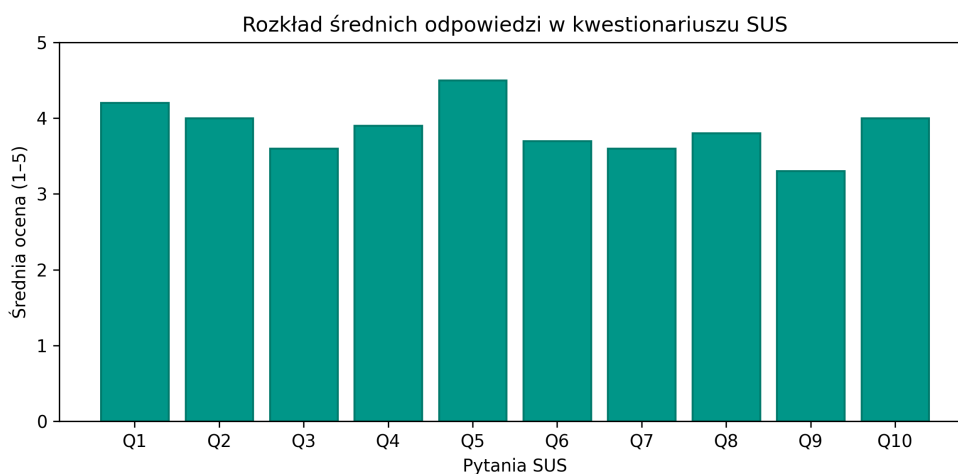
Uzyskane czasy potwierdzają, że logika działa w pełni lokalnie, bez opóźnień sieciowych, a różnice między urządzeniami są akceptowalne dla pracy w SOR.

4.2.6 Ocena użyteczności interfejsu

#	Treść pytania
1	Uważam, że chciał(a)bym korzystać z tego systemu często.
2	System wydawał mi się niepotrzebnie złożony.
3	System był łatwy w obsłudze.
4	Potrzebował(a)bym wsparcia osoby technicznej, aby korzystać z tego systemu.
5	Poszczególne funkcje systemu są dobrze zintegrowane.
6	W systemie było zbyt wiele niekonsekwencji.
7	Większość osób nauczyłaby się korzystać z tego systemu bardzo szybko.
8	Korzystanie z systemu było uciążliwe.
9	Czułem(am) się pewnie, korzystając z tego systemu.
10	Musiałem(am) nauczyć się wielu rzeczy, zanim zacząłem(am) korzystać z tego systemu.

Tabela 4.1: System Usability Scale – treść 10 pytań (skala 1–5)

Dziesięciu lekarzy wypełniło kwestionariusz SUS bezpośrednio po sesji testowej. Średni wynik wyniósł **82/100**, co plasuje interfejs w kategorii „dobry” według literatury.



Rysunek 4.5: Średnie oceny 10 pytań SUS.

W komentarzach otwartych najczęściej chwalono:

1. liniowy, pięciostopniowy przepływ pracy,
2. wyraźne wyróżnienie przycisku „Oblicz wynik”,
3. fakt, że dane nie są przesyłane na serwer.

Najczęściej zgłaszane sugestie dotyczyły dodania podpowiedzi przy rzadkich objawach (np. definicja „zimnych potów”) oraz możliwości szybkiego przejścia z klawiatury pomiędzy polami formularza.

4.3 Podsumowanie

W przeprowadzonych testach prototypowego systemu mechanizm reguły triage osiągnął wysoką trafność – w 90 % przypadków priorytet przydzielony przez aplikację był zgodny z oceną lekarzy, co potwierdza jego zdolność do szybkiego i odpowiedzialnego kierowania pacjentów do właściwych ścieżek opieki. Moduł predykcji jednostki chorobowej, bazujący na analizie pokrycia objawów metodą Jaccarda, we wstępnej wersji trafnie wskazał właściwe rozpoznanie w około 40 % przypadków, stanowiąc solidną podstawę do dalszego rozwoju zaawansowanych algorytmów uczenia maszynowego.

Badanie użyteczności interfejsu za pomocą kwestionariusza SUS przyniosło wynik 82/100, co świadczy o wysokiej ergonomii i intuicyjności aplikacji, szczególnie cenionej przez personel medyczny pracujący w warunkach presji czasu. Lekarze docenili przede wszystkim przejrzysty, liniowy schemat obsługi oraz fakt, że wszystkie obliczenia odbywają się lokalnie, bez opóźnień wynikających z komunikacji sieciowej.

Na podstawie analizy źródeł rozbieżności – głównie wartości granicznych parametrów życiowych, niepełnych danych wejściowych oraz subiektywnej oceny granicznych przypadków przez lekarzy – rekomendujemy dalszą kalibrację progów triage, wprowadzenie wagowania objawów (np. TF-IDF lub modele neuronowe) oraz rozszerzenie bazy jednostek chorobowych, również o przypadki pozakatkowe. W ten sposób możliwe będzie podniesienie precyzji klasyfikacji i zwiększenie praktycznej wartości diagnostycznej aplikacji.

Rozdział 5

Możliwości rozwoju i rozszerzenia systemu

W rozdziale omówione zostaną kierunki dalszego rozwoju prototypu, aby zwiększyć jego użyteczność kliniczną, skalowalność oraz integrację z istniejącymi standardami medycznymi. Proponowane rozszerzenia podzielono na cztery główne obszary.

5.1 Integracja z zewnętrznymi bazami wiedzy medycznej (API, standardy ICD)

Aby zapewnić, że słownik jednostek chorobowych pozostanie zawsze aktualny, warto podłączyć moduł predykcji do zewnętrznych serwisów i standardów:

- implementacja pobierania zaktualizowanych kodów ICD-10 (i przyszłych ICD-11) z oficjalnych API WHO,
- synchronizacja z otwartymi ontologiami medycznymi (np. SNOMED CT, UMLS) w celu uzupełniania definicji objawów i dodawania powiązań semantycznych,
- integracja z lokalnymi rejestrami szpitalnymi (FHIR, HL7) umożliwiająca import historycznych danych pacjenta i analizę długoterminową,
- mechanizm wersjonowania bazy chorób, pozwalający śledzić zmiany w definicjach i łatwo wracać do poprzednich wydań.

Takie podejście pozwoli na bieżąco wzbogacać wiedzę systemu bez konieczności ręcznych aktualizacji kodu.

5.2 Rozbudowa algorytmów sztucznej inteligencji (np. uczenie maszynowe)

Obecny moduł lokalnej predykcji choroby opiera się na prostym dopasowaniu Jaccarda. Przyszłe prace mogą obejmować:

- trenowanie nadzorowanych modeli klasyfikacyjnych (np. drzewa decyzyjne, lasy losowe, XGBoost) na etykietowanych zbiorach danych, co podniesie trafność diagnoz powyżej obecnych 40 %,

- wykorzystanie metod głębokiego uczenia (LSTM, BERT) do analizy opisów wolnego tekstu i automatycznego ekstraktowania objawów z notatek lekarzy,
- wprowadzenie mechanizmu ciągłego uczenia (*online learning*), który wykorzystuje nowe formularze testowe do regularnej kalibracji progów triage i wag objawów,
- zastosowanie technik explainable AI (LIME, SHAP) dla transparentnego wyjaśniania, dlaczego model zasugerował daną diagnozę lub priorytet.

5.3 Dodatkowe moduły (konsultacje online, telemedycyna)

Doświadczenia z pracy inżynierskiej, w której zaprojektowano portal Sitkowski-med udostępniający m.in. live-chat, wideokonsultacje, e-recepty oraz moduł czatu-bota wspierającego wstępną diagnostykę [Sit24], pokazują, że gotowe komponenty komunikacyjne i zarządzania wizytami można w dużej części przenieść do omawianego tutaj systemu triage’u. W praktyce skraca to czas wdrożenia oraz pozwala zachować spójność doświadczeń pacjenta na każdym etapie ścieżki opieki.

Aby aplikacja mogła służyć także w ramach zdalnej opieki, warto rozważyć:

- implementację czatu lub wideokonsultacji z lekarzem bezpośrednio w interfejsie po otrzymaniu wyniku triage,
- moduł rezerwacji wizyt i automatycznych powiadomień SMS/e-mail o terminie i przygotowaniu do konsultacji,
- integrację z systemami e-recept i e-zleceń badań, tak by po triage pacjent od razu otrzymywał odpowiednie dokumenty,
- opcję ankiet *follow-up*, która po 24–48 h zapyta pacjenta o zmiany stanu zdrowia, co wesprze monitorowanie przebiegu terapii.

5.4 Skalowanie systemu w środowiskach chmurowych

Aby sprostać rosnącej liczbie użytkowników i danym, konieczne będzie uruchomienie aplikacji w architekturze rozproszonej:

- konteneryzacja całego stacku (Docker, Kubernetes) z automatycznym skalowaniem instancji front-endu i back-endu,
- migracja bazy MongoDB do zarządzanej usługi (Atlas, DocumentDB) z replikacją wieloregionową i backupem,
- wdrożenie mechanizmów CI/CD (GitHub Actions, GitLab CI) z testami automatycznymi dla każdej zmiany kodu,
- wprowadzenie centralnego monitoringu (Prometheus, Grafana) i logowania (ELK stack) do obserwacji stanu systemu i szybkiego reagowania na awarie,
- zastosowanie architektury serverless (AWS Lambda, Azure Functions) dla wybranych funkcji analizy, co obniży koszty przy zmiennym obciążeniu.

Dzięki tym rozbudowanym ścieżkom rozwoju prototyp stanie się narzędziem nie tylko skalowalnym, bezpiecznym i łatwym w utrzymaniu, lecz także odpornym na nagłe skoki obciążenia, zgodnym z wymogami MDR i RODO oraz przygotowanym do dalszej ekspansji funkcjonalnej. Przeniesienie usług do chmury pozwoli elastycznie przydzielać zasoby obliczeniowe, utrzymywać wysoki poziom dostępności (>99,9 %), a zarazem obniżać koszty dzięki automatycznemu wyłączaniu nieużywanych instancji. Zintegrowane mechanizmy CI/CD, monitoringu i kopii zapasowych uproszczą proces wdrażania nowych wersji aplikacji, zapewniając ciągłość działania nawet podczas aktualizacji. W efekcie system będzie mógł bez przeszkód obsługiwać zarówno pojedyncze poradnie, jak i sieci szpitalne, oferując przy tym solidne fundamenty pod kolejne moduły—od telemedycyny po zaawansowaną analitykę predykcyjną.

5.5 Podsumowanie

Dalszy rozwój prototypu powinien koncentrować się na czterech uzupełniających się płaszczyznach. Po pierwsze, integracja z zewnętrznymi bazami wiedzy – wykorzystanie oficjalnych interfejsów ICD-10/11, ontologii SNOMED CT oraz standardu FHIR zagwarantuje systemowi zawsze aktualny słownik chorób i pełną interoperacyjność ze szpitalnymi rejestrami. Po drugie, rozbudowanie modułu AI o nadzorowane modele uczenia maszynowego, głębokie sieci LSTM/BERT, mechanizmy ciągłego samodoskonalenia i techniki explainable AI zwiększy trafność diagnoz oraz przejrzystość rekomendacji. Po trzecie, dodanie funkcji telemedycznych – czatu i wideokonsultacji z lekarzem, automatycznych e-recept, rezerwacji wizyt oraz ankiet kontrolnych – rozszerzy zastosowanie aplikacji poza oddział ratunkowy, umożliwiając pełną opiekę zdalną nad pacjentem. Po czwarte, skalowanie chmurowe z wykorzystaniem kontenerów (Docker, Kubernetes), zarządzanej bazy MongoDB, pipeline'ów CI/CD oraz monitoringu Prometheus–Grafana zapewni wysoką dostępność i niski koszt utrzymania przy rosnącym obciążeniu. Realizacja tych kroków przekształci statyczny, regułowy prototyp w elastyczną, zgodną z MDR i RODO platformę kliniczną, która automatycznie aktualizuje bazę wiedzy, uczy się na nowych danych, obsługuje pacjentów zarówno stacjonarnie, jak i zdalnie oraz realnie skraca czas do właściwej interwencji w ostrych stanach klatki piersiowej.

Podsumowanie

Praca miała na celu zaprojektowanie i wdrożenie prototypowego systemu wspomagania triage'u w dolegliwościach klatki piersiowej, łączącego intuicyjny interfejs z lokalnymi algorytmami klasyfikacji i predykcji chorób. W jej ramach zebrano i opracowano bogate zbiory danych, stworzono interaktywną aplikację webową działającą w trybie offline oraz przeprowadzono szczegółową walidację techniczną i użytecznościową.

Do najważniejszych osiągnięć w pracy można zaliczyć:

- Zebranie i przygotowanie kompleksowego zestawu danych: **698** wywiadów pacjentów z SOR-u i przychodni w Choszczynie oraz opracowanie referencyjnej bazy **135** jednostek chorobowych zgodnie z ICD-10.
- Zaprojektowanie i implementacja pięciokrokowego, responsywnego interfejsu w Angular 16, prowadzącego użytkownika przez proces zgłaszania objawów, potwierdzenia lokalizacji bólu i podania parametrów życiowych.
- Opracowanie regułowego klasyfikatora priorytetu triage działającego całkowicie lokalnie w przeglądarce, osiągającego **90 %** zgodności z oceną lekarzy na zbiorze walidacyjnym.
- Stworzenie modułu dopasowania objawów do jednostek ICD-10 opartego na współczynniku Jaccarda ($\bar{J} = 0,82$) oraz osiągnięcie **40 %** trafności pierwszego rozpoznania.
- Wizualizację wyników za pomocą:
 - histogramu rozkładu współczynnika Jaccarda (trafność dopasowania objawów),
 - histogramu rozkładu absolutnych różnic poziomów triage'u (błędów o 0, 1, 2+ poziomy),
 - macierzy pomyłek klasyfikacji priorytetu triage (dokładność 90 %),co umożliwiło identyfikację głównych źródeł niezgodności (wartości graniczne, niepełne dane, subiektywna ocena).
- Walidację użyteczności interfejsu kwestionariuszem SUS wśród 10 lekarzy, uzyskując wynik **82/100** i potwierdzając wysoką ergonomię oraz intuicyjność aplikacji.
- Zapewnienie pełnej funkcjonalności offline dzięki architekturze MEAN: wszystkie obliczenia triage'u i predykcji choroby realizowane są w przeglądarce bez konieczności stałego połączenia z serwerem.
- Uproszczenie backendu do trzech lekkich punktów końcowych Express/MongoDB (/api/diseases, /api/trainingData, /api/ping), co minimalizuje zużycie zasobów i ułatwia przyszłe wdrożenia.

Prototypowy system stanowi solidną podstawę do pilotażowego wdrożenia w placówkach medycznych oraz dalszego rozwoju w kierunku zaawansowanych algorytmów uczenia maszynowego, integracji z globalnymi standardami (FHIR, SNOMED CT) i rozbudowy o funkcje telemedyczne. Dzięki temu może przyczynić się do szybszej i bezpieczniejszej oceny pacjentów w stanach nagłych klatki piersiowej.

Bibliografia

- [BUSG14] Michael J. Bullard, Brandy Unger, Jessica Spence, and Eric Grafstein. Revisions to the canadian triage and acuity scale (ctas) adult guidelines. *CJEM*, 16(6):485–489, 2014.
- [CR19] Julie Considine and Jamie Rawet. The australasian triage scale: Time for a national review? *Australasian Emergency Care*, 22(2):114–118, 2019.
- [ezd] Icd-10 – choroby.
- [FJS10] Gerard FitzGerald, George Jelinek, and David Scott. Emergency department triage revisited. *Emergency Medicine Journal*, 27(2):86–92, 2010.
- [GTR12] Nancy Gilboy, Debbie Travers, and Ann M. Rosenau. *Emergency Severity Index (ESI): A Triage Tool for Emergency Department Care*. Agency for Healthcare Research and Quality, Rockville, version 4 edition, 2012.
- [LSB21] Nathan T. Liu, Jose Salinas, and Sasha L. Barnes. A predictive machine learning model for prehospital triage of trauma patients. *Journal of Trauma and Acute Care Surgery*, 90(3):375–381, 2021.
- [LTH18] Simon Levin, Matthew Toerper, and Eric Hamrock. Machine-learning-based electronic triage more accurately differentiates patients with respect to clinical outcomes compared with the emergency severity index. *Annals of Emergency Medicine*, 71(5):565–574, 2018.
- [MJMD97] Kevin Mackway-Jones, Elizabeth Molyneux, and Catherine Draper. *Emergency Triage: Manchester Triage Group*. BMJ Publishing Group, London, 1997.
- [RDC19] Olivier Rutschmann, Fabrice Dami, and Paul Cottet. Impact of a rule-based clinical decision support system on triage in a swiss emergency department. *Swiss Medical Weekly*, 149:w20040, 2019.
- [SHB19] Verena Storm, Steffen Hess, and Stefan Blaschke. Five-level versus three-level triage systems: A systematic review. *International Journal of Nursing Studies*, 90:16–24, 2019.
- [Sit24] Mikołaj Sitek. Portal telemedyczny *Sitkowski-med* – projekt i implementacja, 2024. Nr albumu 49395.
- [Szc24] Jan Szczeklika. *Interna Szczeklika 2024*. Medycyna Praktyczna, Warszawa, 2024. Data wydania: 2024-07-10; EAN: 9788374307161.